

RELATIONAL DATABASE MANAGEMENT SYSTEM - I

Theory by Maitri Hingu



NOVEMBER 1, 2020

1. What is Data?

"Data is raw facts which can be processed later to gain or generate information."

Example: Mother, Market.

Thus, Data could be defined.

2. What is Information?

"Processed data is known as information."

Example: My mother goes to market.

Thus, Information could be defined.

3. What is Database?

"The collection or set of interrelated data is known as database. It contains information about one enterprise."

Example: University Database, Online Shopping Database.

Thus, Database could be defined.

4. What is Database Management System (DBMS)?

"The collection of interrelated data (DB) and set of Program to access or manipulate those data (MS) is known as Database Management System (DBMS)."

Example: FoxPro, MS Access

Thus, Database Management System could be defined.

MAITRI K. HINGU

5. What is Relational Database Management System (RDBMS)?

"The RDBMS is a DBMS which is based on the relational model introduced by Dr. E. F. Codd. RDBMS stores data in form of related tables. It must satisfy the all 12 rules of Dr. E. F. Codd, but in reality, there is no DBMS which satisfy the all 12 rules so if it satisfies more than 7 to 8 rules then DBMS becomes RDBMS."

Example: MS SQL, MySQL, Oracle, IBM DB2, PostgreSQL.

Thus, Database Management System could be defined.

6. What is Instance?

"The collection of information stored in the database at a one particular moment is known as instance of that database."

Thus, Instance could be defined.

7. What is Schema? Explained its types in detail.

"The overall structure or design of the database is known as the schema of that database."

There are three types of schema.

MAITRI K. HINGU

1) Physical Schema:

Physical schema is a term used in database management to describe how data is to be represented and stored in secondary storage using a database management system (DBMS).

2) Logical Schema:

A logical schema is a design-centric database structure built to meet your business requirements. It is a model that exists on a white board or in a diagraming tool. It is like the architect's drawings of your database.

3) Sub-Schema (View level Schema):

A subschema is a subset of the schema and inherits the same property that a schema has. The schema for a view is often called sub-schema. Sub-schema refers to an application programmer's view of the data item types and record types, which he or she uses.

Thus, Schema and its types could be explained.

MAITRI K. HINGU

8. What is Data Abstraction? Explain its level also.

OR

Explain View of Data.

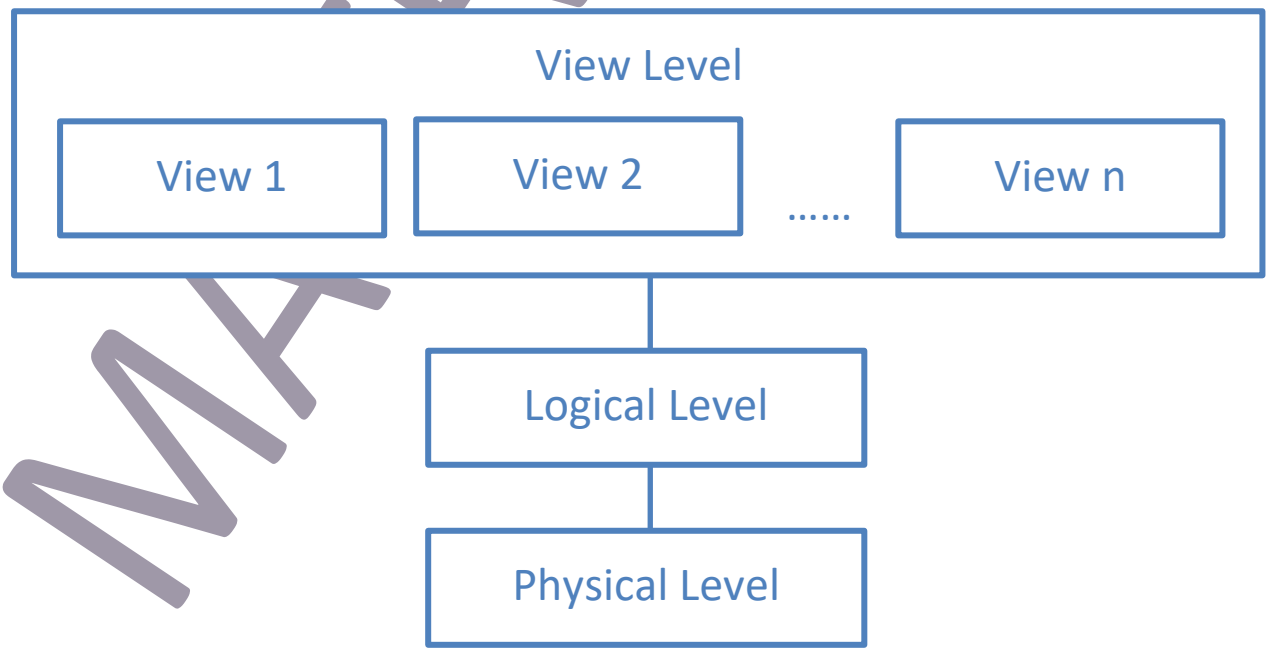
A Database Management System is collection of interrelated data and set of programs that allow user to access and modify those data. A major purpose of database management system is to provide users abstract view of data.

Data Abstraction:

"Hiding the complexity from the database users is known as data abstraction."

The system hides certain details of how data is stored and maintained.

There are three level of data abstraction which can be demonstrated as below.



MAITRI K. HINGU

1) Physical Level Data Abstraction:

This is lowest level of data abstraction which describes "How data is actually stored in secondary storage?" It describes complex low-level data structures in detail.

2) Logical Level Data Abstraction:

This is next higher level of data abstraction which describes "what data are stored in the database?" and "what relationship exists between those data?"

3) View Level Data Abstraction:

This is highest level of data abstraction which describes the user interaction with database system.

Thus, Data Abstraction and its level could be explained.

9. What is Entity?

"Things or objects which are distinguishable from the real world are known as entity."

Example: Each student is an entity. Entities are described in a database by a set of attributes like Roll no, Name, Address, etc.

Thus, Entity could be defined.

MAITRI K. HINGU

10. What is Entity Set?

"The set of all entities of the same type is called as entity set."

Thus, Entity set could be defined.

11. Explain Types of Entity.

There are five types of Entity in DBMS.

1) Strong Entity:

Entity which has key attribute in its attribute list to uniquely identify its each row is known as Strong Entity.

Example:

EmployeeMst (EID, FName, MName, LName, Address, Contact, EmailID, Salary, DOJ)

2) Weak Entity:

Entity which does not have key attribute in its attribute list to uniquely identify its each row is known as Weak Entity.

Example:

EmployeeMst (FName, MName, LName, Address, Contact, EmailID, Salary, DOJ)

MAITRI K. HINGU

3) Recursive Entity:

It is also called Self Referential Relationship Entity Type. It is an entity type with foreign key referencing to same table or itself. Recursive Entity Type occurs in a unary relationship. Generally, relationship has always been between two different entities. However, it is possible for the same entity to participate in the relationship twice that is known as recursive relationship or self-join or self-reference.

Example:

An employee who is also a manager but a manager is also an employee, whose details will be held in the employee entity. To implement this, a foreign key of the employee's manager number would be held in each employee record.

EmployeeMst (EID, FName, MName, LName, Address, Contact, EmailID, Salary, DOJ, MID (F.K. of EID))

4) Composite Entity:

If a Many – to – Many relationship exists then we must eliminate it by using composite entities. Composite Entities are the entities which exist as a relationship as well as an entity. The many

MAITRI K. HINGU

– to – many relationship will be converted to one – to – many relationship.

Composite Entities are also called Bridge Entities, because they act like a bridge between the two entities which have many – to – many relationship. Bridge or Composite entity composed of the primary keys of each of the entities to be connected. It is also known as relationship table or junction table.

A composite entity is represented by a rhombus shape within a rectangle in an E-R Diagram.

Example:

DoctorMst (DOCID, FName, MName, LName, Address, Contact, City, State, Country, SpecialistIn, Fees, OtherHospital)

PatientMst (PID, FName, MName, LName, Address, Contact, City, State, Country)

DoctorPatientDetail (DPID, DOCID, PID, DOA, DOD, Decease, Symptoms, Treatment, TotalAmt)

Here, DoctorPatientDetail is Composite Entity.

MAITRI K. HINGU

5) Super Type Entity and Sub Type Entity:

Super Type Entity is an entity which has relationship with one or more sub types and it contains attributes that are common to its Sub type.

Sub Type Entity is sub group of super type entity and has unique attributes, but they will be different from each sub type entity.

Example:

Super Type Entity: PersonMst (PID, FName, LName, MName, Address, Contact, DOB)

Sub Type Entity: EmployeeMst (EID, FName, LName, MName, Address, Contact, DOB, DOJ, Salary, Department)

StudentMst (SID, FName, MName, LName, Address, Contact, DOB, DOA, Mark1, Mark2, Total, Percent, Class)

Thus, Types of Entity could be explained.

MAITRI K. HINGU

12. What is Attribute?

"An attribute is a characteristic of an entity by which an entity can be identified. It is a synonym of the column."

Example: For Employee Entity EID, FName, MName, LName, Address, City, State, Country are Attributes.

Thus, Attribute could be defined.

13. What is Attribute Set?

"Set or Collection of all attributes of one entity is known as attribute set."

Example: For Employee Entity EID, FName, MName, LName, Address, City, State, Country are Attribute set.

And for Department Entity DID, Dname, DCode are Attribute set.

Thus, Attribute set could be defined.

MAITRI K. HINGU

14. Explain Types of Attribute.

There are 11 types of Attribute in DBMS.

1) Single Valued Attribute:

An attribute that has a single value for an entity is known as single valued attribute.

Example: Age of Employee.

2) Multi Valued Attribute:

An attribute that may have multiple values for the same entity is known as multi valued attribute.

Example: One Employee can have more than one address.

3) Compound Attribute or Composite Attribute:

Attribute can be subdivided into two or more Attribute is known as Compound or Composite Attribute.

Example: Full Name of employee can be divided into First name, Middle name and Last name.

4) Simple Attributes or Atomic Attributes:

The attribute which cannot be divided into smaller subparts is known as Simple or Atomic attribute.

Example: First name of employee.

MAITRI K. HINGU

5) Stored Attribute:

An attribute which cannot be derived from another attribute is known as stored attribute.

Example: DOB of employee.

6) Derived Attribute:

Attributes derived from other stored attribute is known as derived attributes.

Example: Employee's age can be derived from Date of Birth and Today's date.

7) Complex Attribute:

If an attribute for an entity, is built using composite and multivalued attributes, then these attributes are called complex attributes.

Example: An address attribute for any Entity can be made up of HouseNo, SocietyName, Area, LandMark, City, State, Country, Pincode. Generally, we give TEXT datatype to it but not each one is going to follow the structure of address shown above. For that, we must create own datatype and need to assign it to address attribute.

```
CREATE OR REPLACE TYPE AddressType AS OBJECT
(HouseNo NUMBER NOT NULL,
 SocietyName VARCHAR (50) NOT NULL,
 Area VARCHAR (30) NOT NULL,
```

MAITRI K. HINGU

```
LandMark VARCHAR (30) NOT NULL,
City VARCHAR (30) NOT NULL,
State VARCHAR (30) NOT NULL,
Country VARCHAR (30) NOT NULL,
Pincode VARCHAR (10) NOT NULL);
```

8) Key Attributes or Prime Attributes:

An attribute which has distinct value for each entity in an entity set is known as Key Attribute or Prime Attribute. Generally Primary key, Unique Key, Super Key, Candidate Key or Minimal Super Key, Minimal Candidate Key in the table becomes Key Attribute or Prime Attribute.

Example: Roll number in a Student Entity Type.

9) Non-Key Attributes or Non-Prime Attributes:

These are attributes other than candidate key attributes in a table.

Example: City is a non-key attribute as it does not represent the main characteristics of the entity.

10) Required Attributes or Not Null Attributes:

A required attribute is an attribute that must have a data value. These attributes are required because they describe what is important in the entity.

MAITRI K. HINGU

Example: In a StudentMst entity, FirstName and LastName are required attribute.

11) Optional Attributes or Null Attributes:

An optional attribute may have a value in it or can be left blank.

Example: In a StudentMst entity, EmailID is an optional attribute as some students may not have EmailID.

Thus, Types of attribute could be explained.

15. What is Relationship?

"Association among the entities is known as relationship."

Example: A customer_account relationship associates a customer with each account he or she has.

Thus, Relationship could be defined.

16. What is Relationship set?

"Relationship set is a set of relationships of the same type."

Thus, Relationship set could be defined.

MAITRI K. HINGU

17. Explain Types of Relationship?

"According to Mapping cardinality there are 4 types of relationship."

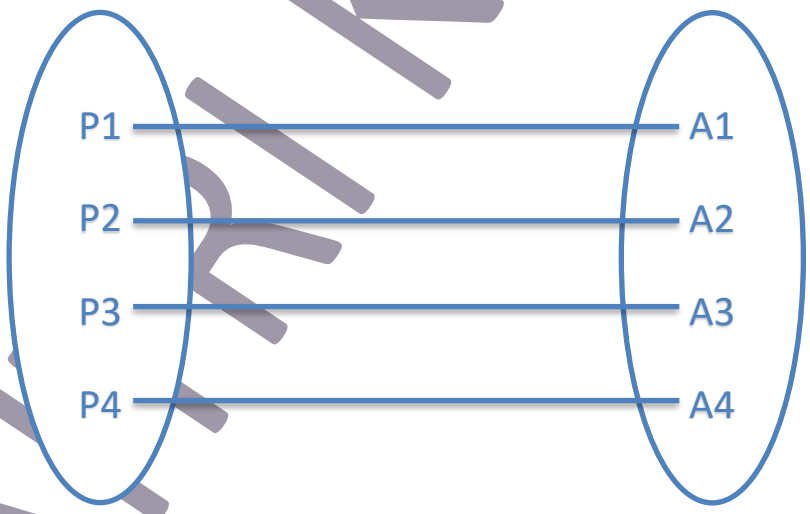
1) One – to – One:

An entity in A is associated with at most one entity in B, and an entity in B is associated with at most one entity in A.

Example: One Person can have only one Aadhar number (UID)

PersonMst – to – AadharMst

One – to – One



2) One – to – Many:

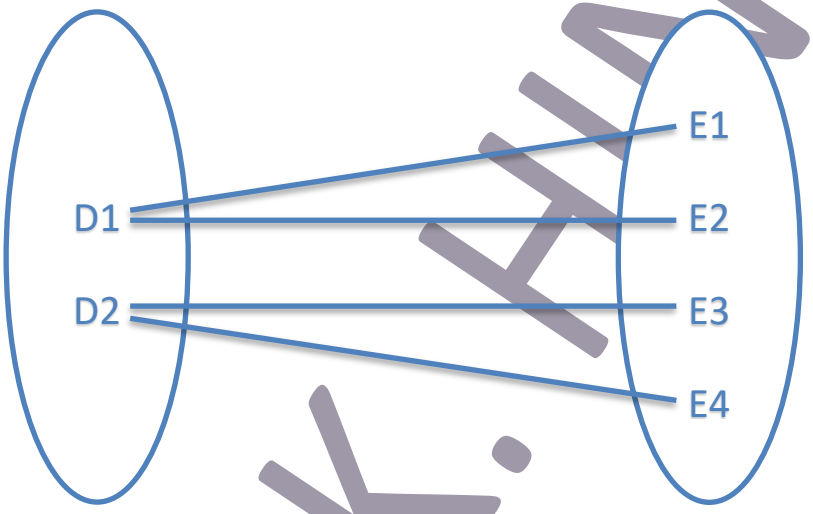
An entity in A is associated with any number in B. An entity in B is associated with at most one entity in A.

MAITRI K. HINGU

Example: One Department can have more than one employee in it.

DepartmentMst – to – EmployeeMst

One – to – Many



3) Many – to – One:

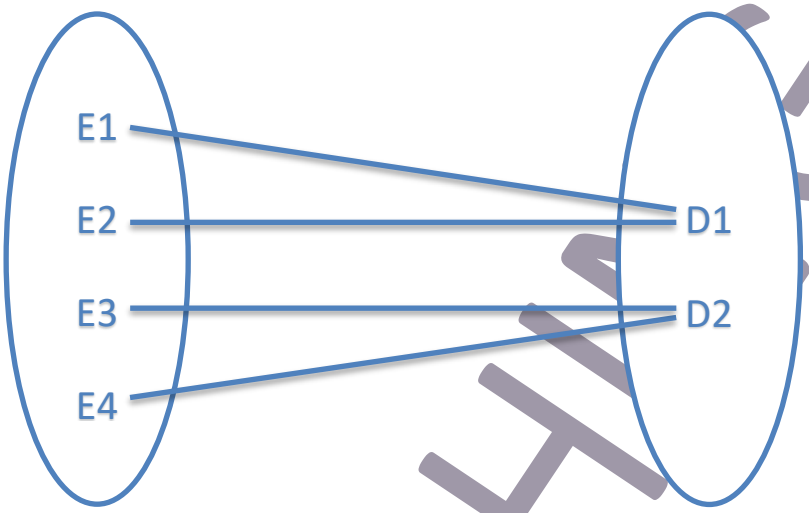
An entity in A is associated with at most one entity in B. An entity in B is associated with any number in A.

Example: Many Employee can work in one department.

MAITRI K. HINGU

EmployeeMst – to – DepartmentMst

Many – to – One



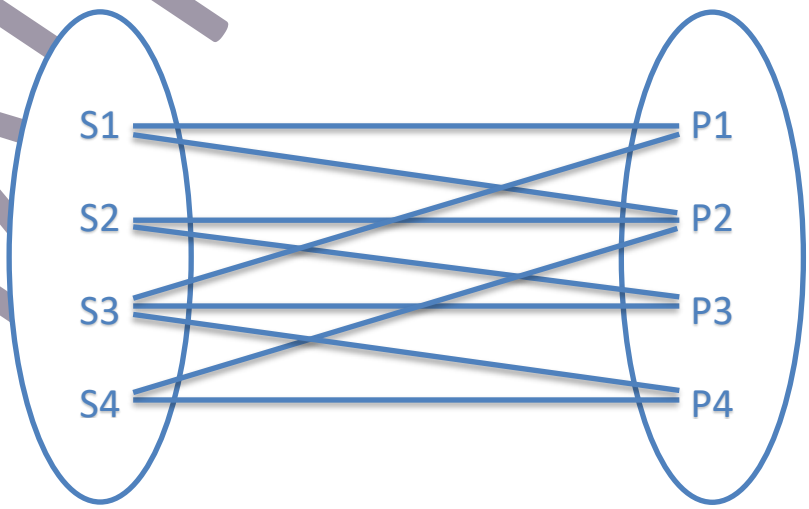
4) Many – to – Many:

Entities in A and B are associated with any number from each other.

Example: One student learns from many professor and one professor can teach many students.

StudentMst – to – ProfessorMst

Many – to – Many



MAITRI K. HINGU

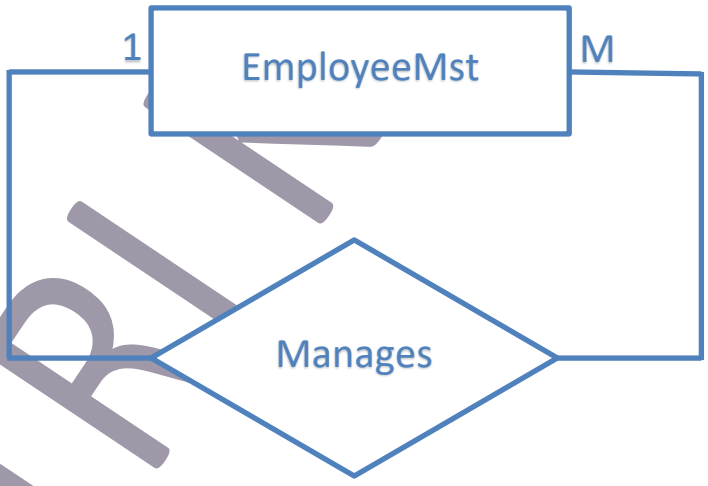
According to E- R Model there are 4 types of relationship.

1) Unary Relationship:

If there is only one entity which is involve in relationship, then it is known as unary relationship.

Example:

An employee who is also a manager but a manager is also an employee, whose details will be held in the employee entity.



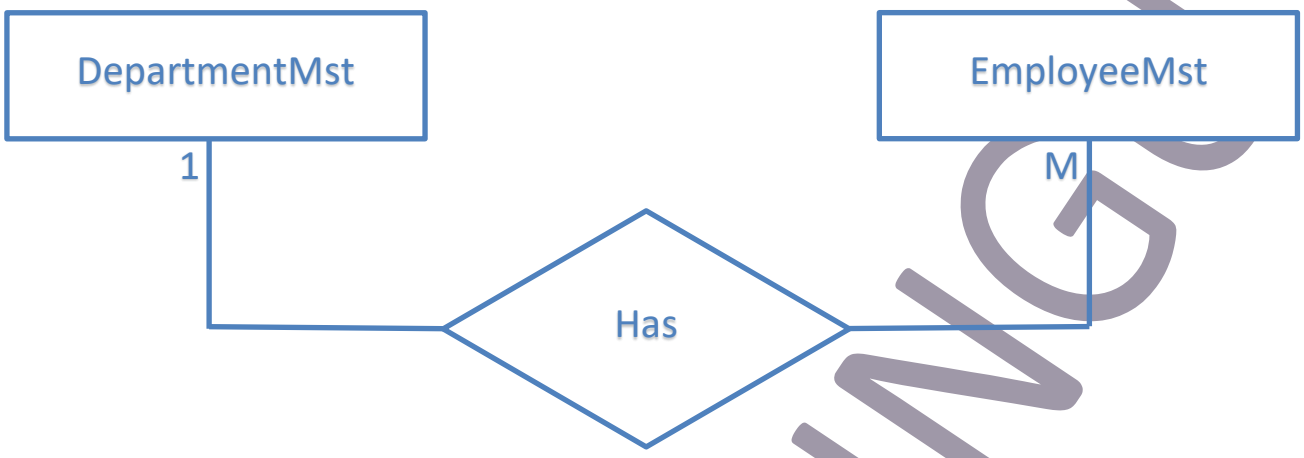
2) Binary Relationship:

If there are two entity which are involve in relationship, then it is known as binary relationship.

Example:

In one department, many employees can work. So, One Department Many Employee.

MAITRI HINGU



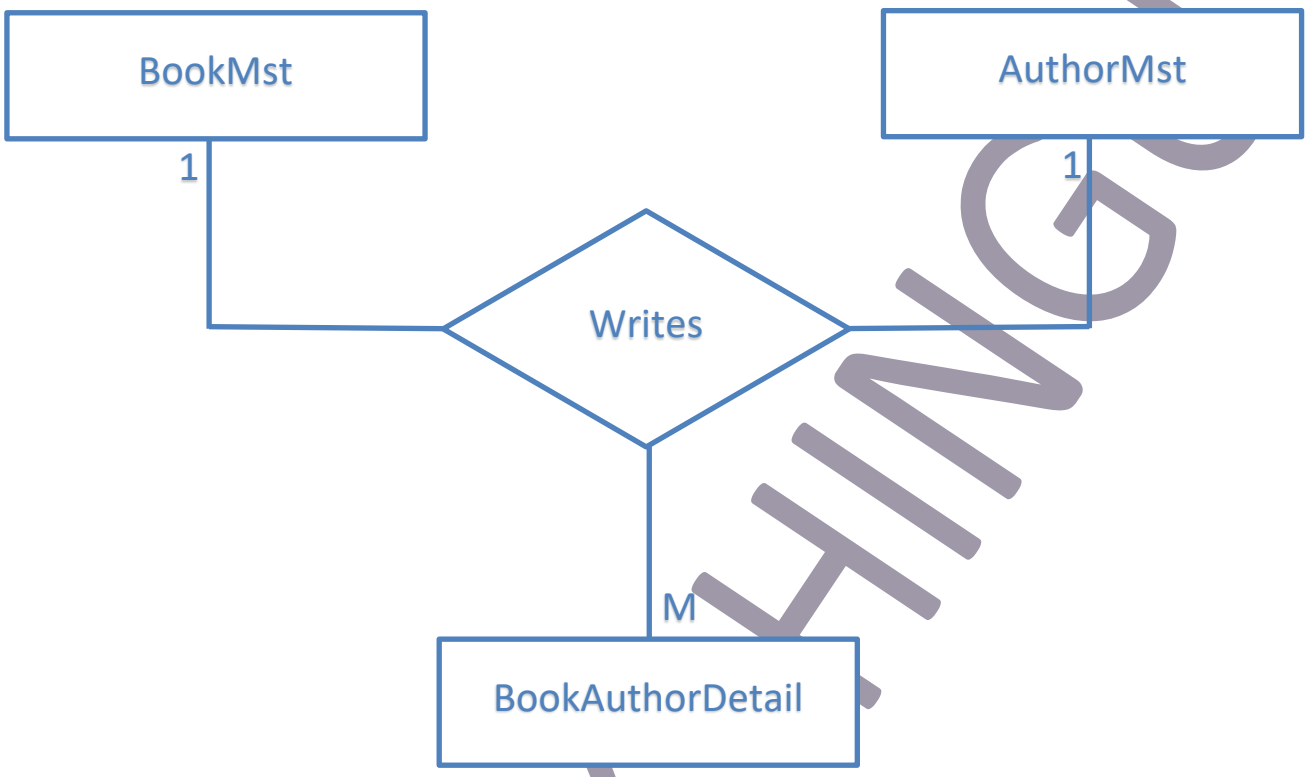
3) Ternary Relationship:

If there are three entity which are involve in relationship, then it is known as ternary relationship.

Example:

One book can be written by many employees where as one author can also write many book throughout his life. So, Many to Many Relationship will be there.

MAITRI K. HINGU



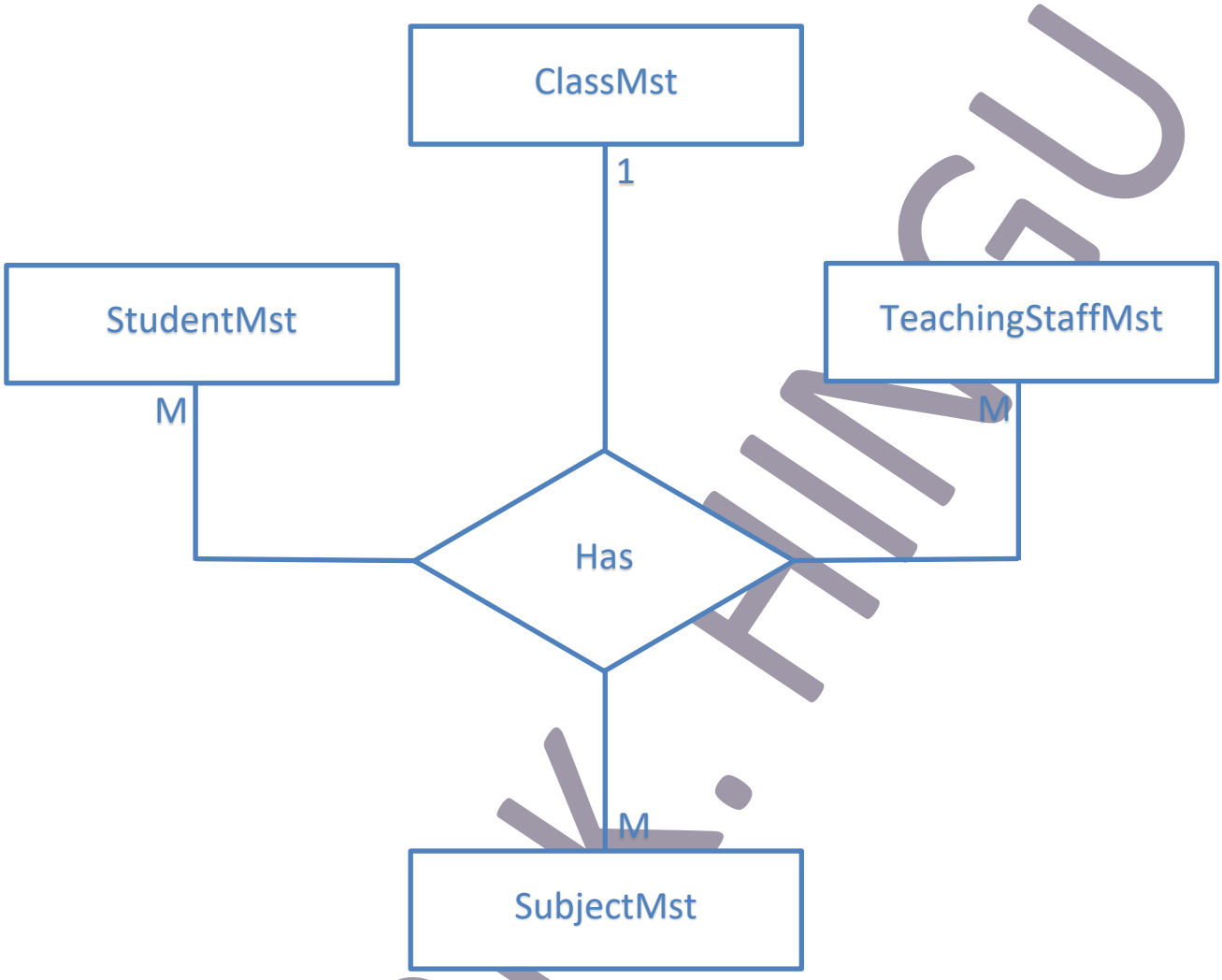
4) Quaternary Relationship:

If there are four entity which are involve in relationship, then it is known as quaternary relationship.

Example:

In one class, many students learns many subjects which is taught by many teaching staff. So, One Class Many Student, Many Student Many Subject, Many Student Many Teaching staff and Many Teaching Staff Many Subject.

MAITRI K. HINGU



Thus, Types of Relationship could be explained.

MAITRI K. HINGU

18. What is mapping Cardinality? Explain it in Detail.

Mapping Cardinality is used to express the number of entities to which another number of entities can be associated via a relationship.

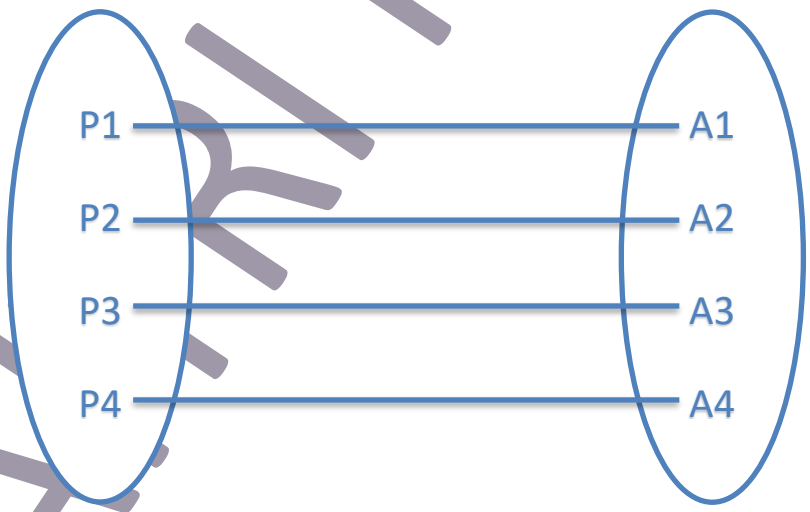
1) One – to – One:

An entity in A is associated with at most one entity in B, and an entity in B is associated with at most one entity in A.

Example: One Person can have only one Aadhar number (UID)

PersonMst – to – AadharMst

One – to – One



MAITRI K. HINGU

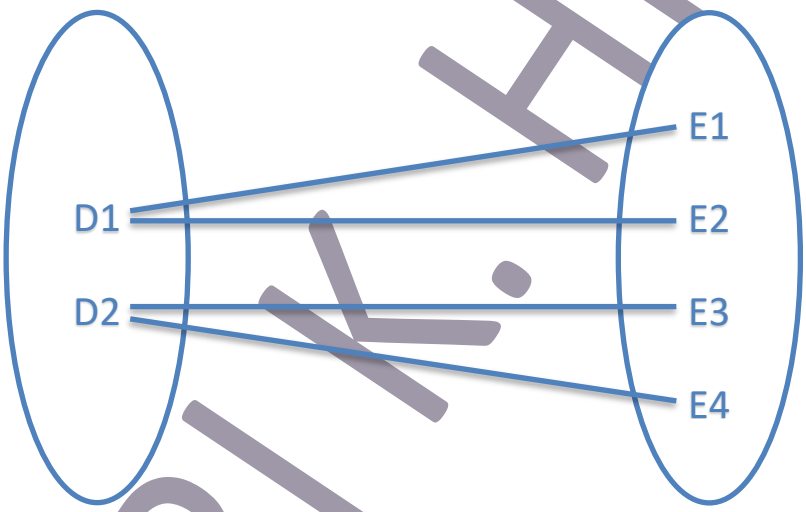
2) One – to – Many:

An entity in A is associated with any number in B. An entity in B is associated with at most one entity in A.

Example: One Department can have more than one employee in it.

DepartmentMst – to – EmployeeMst

One – to – Many



3) Many – to – One:

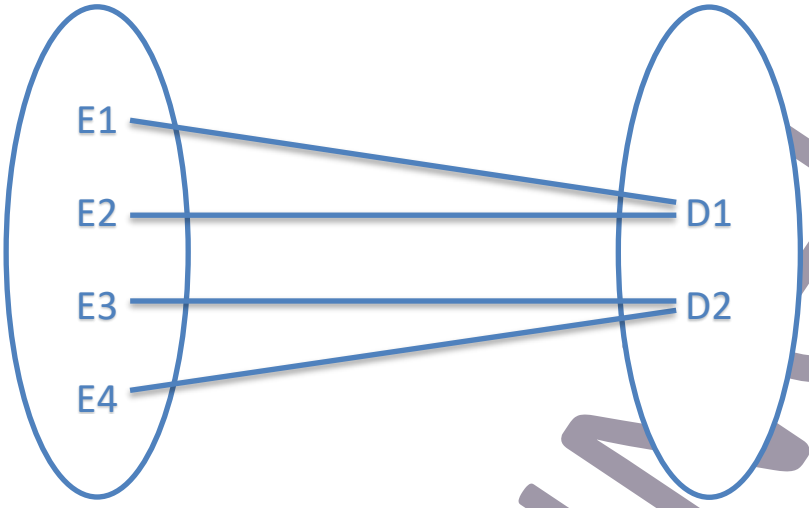
An entity in A is associated with at most one entity in B. An entity in B is associated with any number in A.

Example: Many Employee can work in one department.

EmployeeMst – to – DepartmentMst

Many – to – One

MAITRI K. HINGU



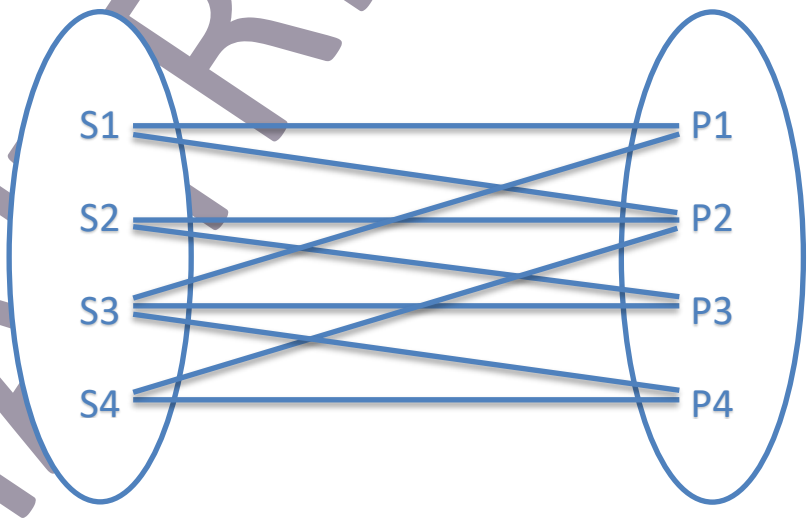
4) Many – to – Many:

Entities in A and B are associated with any number from each other.

Example: One student learns from many professor and one professor can teach many students.

StudentMst – to – ProfessorMst

Many – to – Many



Thus, Mapping Cardinality could be explained.

MAITRI K. HINGU

19. Synonyms of the terms use in DBMS.

TABLE	Relation, Entity Set, Record Set, Class
ROW	Tuple, Entity, Record, Object, Instance
COLUMN	Field, Attribute, Property, Data Member

Thus, Synonyms could be stated.

20. What is Data Redundancy?

"Data Redundancy means same data is being stored in multiple places without necessity whether in same table or different. It simply means data repetition or data duplication."

Thus, Data Redundancy could be explained.

21. What is Data Inconsistency?

"Data Inconsistency is a situation which arise because of same data is stored in database and when any changes occur in one place user fails to change them in another one which leads to the data inconsistency."

Thus, Data Inconsistency could be explained.

MAITRI K. HINGU

22. What is Data Integrity?

"Data Integrity refers to the accuracy and consistency of data, stored in a database."

Thus, Data Integrity could be explained.

23. What is Constraint?

Constraints are the rules enforced on data columns of table. These are used to limit the type of data that can go into the table. This ensures the accuracy and reliability of the data in the database.

There are five types of I/O constraints.

1) Primary Key:

It must require value and it does not accept duplicate value.

Example:

```
CREATE TABLE DepartmentMst
(DID Number PRIMARY KEY,
DName Text(20) NOT NULL);
```

2) Foreign Key:

It may require value or not. But if you put value in this column it must have been in its parent table. You can only give foreign key on Primary key or unique key of another table.

MAITRI K. HINGU

Example:

```
CREATE TABLE EmployeeMst
(EID NUMBER PRIMARY KEY,
 FName TEXT(10) NOT NULL,
 LName TEXT(10) NOT NULL,
 Address TEXT(60) NOT NULL,
 DOB DATE NOT NULL,
 DOJ DATE NOT NULL,
 Salary NUMBER NOT NULL,
 DID NUMBER REFERENCES DepartmentMst(DID) );
```

3) Not null:

It must require value whether it is duplicate or not it is not a problem.

Example:

```
CREATE TABLE DepartmentMst
(DID NUMBER PRIMARY KEY,
 DName TEXT(10) NOT NULL);
```

4) Check:

It is used to check condition on column's value during insertion and updating column value.

Example:

```
CREATE TABLE DepartmentMst
(DID NUMBER PRIMARY KEY,
 DCode TEXT(5) NOT NULL CHECK(DCode LIKE 'D*'),
 DName TEXT(10) NOT NULL);
```

MAITRI K. HINGU

5) Unique Key:

It may require value or not. But if you are inserting or updating value of column it must be unique.

Example:

```
CREATE TABLE DepartmentMst
(DID NUMBER PRIMARY KEY,
 DName Text(10) UNIQUE);
```

Thus, I/O Constraints could be explained.

Other Integrity Constraints are,

1) Domain Integrity:

Domain integrity means the definition of a valid set of values for an attribute. You define,

- data type
- length or size
- is null value allowed?
- is the value unique or not?

for an attribute.

You may also define the default value, the range (values in between) and/or specific values for the attribute.

These definitions ensure that a specific attribute will have a right and proper value in the database.

MAITRI K. HINGU

2) Referential Integrity:

The referential integrity constraint is specified between two tables and it is used to maintain the consistency among rows between the two tables.

The rules are:

- You can't delete a record from a primary table if matching records exist in a related table.
- You can't change a primary key value in the primary table if that record has related records.
- You can't enter a value in the foreign key field of the related table that doesn't exist in the primary key of the primary table.

However, you can enter a Null value in the foreign key, specifying that the records are unrelated.

3) Entity Integrity:

The entity integrity constraint states that primary keys can't be null. There must be a proper value in the primary key field.

This is because the primary key value is used to identify individual rows in a table. If there were null values for primary keys, it would mean that we could not identify those rows.

On the other hand, there can be null values other than primary key fields. Null value means that one

MAITRI K. HINGU

doesn't know the value for that field. Null value is different from zero value or space.

The entity integrity constraints assure that a specific row in a table can be identified.

4) Assertions:

An assertion is any condition that the database must always satisfy. Domain constraints and referential integrity constraints are also special form of assertion. However, there are many constraints that we cannot express by using only these special forms. For Example, "Every Department must have at least five employees" must be expressed as assertion. When assertion is created, system test it for validity. If the assertion is valid, then any future modification is allowed only if it does not cause that assertion to be violated.

5) Authorization:

Authorization is the process of giving someone permission to do or have something. In multi-user computer systems, a system administrator defines for the system which users are allowed access to the system and what privileges are given to them. Like, reading database (SELECT), writing into database (INSERT), changing database (UPDATE), removing data from the database (DELETE) which are also known as READ

MAITRI K. HINGU

AUTHORIZATION, INSERT AUTHORIZATION, UPDATE AUTHORIZATION and DELETE AUTHORIZATION.

We may assign user all, none or combination of these authorization.

Thus, Integrity Constraints could be explained.

24. Who is DBA or Database Manager or Database Administrator?

"The database administrator is a person having central control over data and programs for accessing that data."

Thus, DBA could be defined.

25. What is Query?

"Queries are the main way to make a request for information from a database. Queries consist of questions presented to the database in a predefined format, in most cases SQL (Structured Query Language) format."

Thus, Query could be defined.

26. What is Nested Query or Sub Query?

"When we write query within query is known as nested query or sub query."

Example: Display those employees' detail who works in HR department.

MAITRI K. HINGU

```
SELECT *
FROM EmployeeMst
WHERE DID IN (SELECT DID
                FROM DepartmentMst
                WHERE DName = 'HR');
```

Thus, Nested Query could be defined.

27. What is SQL?

"SQL stands for Structured Query Language. SQL is used to communicate with a database. According to ANSI (American National Standards Institute), it is the standard language for relational database management systems."

SQL is divided into another four language.

1) DDL (Data Definition Language)
2) DML (Data Manipulation Language)
3) DCL (Data Control Language)
4) TCL (Transaction Control Language)

Thus, SQL could be defined.

28. What is Data Definition Language (DDL)?

"Database schema is expressed by a special language is called Data Definition Language (DDL). DDL is set of commands, used to create, alter, drop and truncate table but not the data."

MAITRI K. HINGU

Commands: CREATE TABLE, ALTER TABLE, DROP TABLE, and TRUNCATE TABLE.

1) CREATE TABLE:

Usage: It is used to create the table.

Syntax:

```
CREATE TABLE tableName
(field1 datatype[(size)] [CONSTRAINT]
  [, field2 datatype[(size)] [CONSTRAINT], …….. ,
     fieldn datatype[(size)] [CONSTRAINT]]
);
```

Example:

```
CREATE TABLE DepartmentMst
(DID NUMBER PRIMARY KEY,
 DName TEXT(10) NOT NULL
);
```

2) ALTER TABLE:

Usage: It is used to alter or change structure of the table.

Syntax:

1. For Adding a column

```
ALTER TABLE tableName
ADD (field1 datatype[(size)], ….,
        fieldn datatype[(size)]);
```

MAITRI K. HINGU

2. For altering a column constraint, size and datatype

ALTER TABLE tableName
MODIFY field datatype[(size)] [CONSTRAINT];

3. For deleting a column

ALTER TABLE tableName
DROP COLUMN field;

Example:

a) ALTER TABLE DepartmentMst
ADD (DCode TEXT(3));

b) ALTER TABLE DepartmentMst
MODIFY DCode TEXT(5) NOT NULL;

c) ALTER TABLE DepartmentMst
DROP COLUMN DCode;

3) DROP TABLE:

Usage: It is used to drop or delete the table.

Syntax:

DROP TABLE tableName;

Example:

DROP TABLE DepartmentMst;

MAITRI K. HINGU

4) TRUNCATE TABLE:

Usage: It is used to drop or delete the table and again recreate the same table with same name and same constraint.

Syntax:

TRUNCATE TABLE tableName;

Example:

TRUNCATE TABLE DepartmentMst;

Thus, DDL could be explained.

MAITRI K. HINGU

29. What is Data Manipulation Language (DML)?

"Data in Database is managed by a special language is called Data Manipulation Language (DML). DML is set of command, used to insert, update, delete and select data."

Commands: INSERT INTO, UPDATE, DELETE, SELECT.

1) INSERT INTO:

Usage: It is used to insert record in the table.

Syntax:

```
INSERT INTO tableName[(field1, field2,……,fieldn)]
VALUES(value1[, value2, ……, valuen]);
```

Example:

```
INSERT INTO DepartmentMst(DID, Dname)
VALUES (1, 'Finance');
```

2) UPDATE:

Usage: It is used to update or change existing value of column in the table.

Syntax:

```
UPDATE tableName
SET field1=newValue1 [, field2=newValue2, ……,
    Fieldn=newValuen]
[WHERE condition];
```

MAITRI K. HINGU

Example:

```
UPDATE DepartmentMst
SET DName='HR'
WHERE DID=1;
```

3) DELETE:

Usage: It is used to delete one or more row from the table.

Syntax:

```
DELETE
FROM tableName
[WHERE condition];
```

Example:

```
DELETE
FROM DepartmentMst
WHERE DID=5;
```

4) SELECT:

Usage: It is used to fetch or select or display one or more row from the table accordingly condition, group or order.

Syntax:

```
SELECT { * | [DISTINCT] field1 [, field2, ……, filedn] |
Aggregate Function}
FROM tableName
[WHERE condition]
```

MAITRI K. HINGU

[ORDER BY field1 {[ASC]|DESC} [, field2 {[ASC]|DESC}, ……, filedn {[ASC]|DESC}]]
[GROUP BY field1 [, field2, ……, filedn]
[HAVING condition]]
[FETCH FIRST n ROWS ONLY]; //used instead of TOP n

Example:

```
SELECT *
FROM DepartmentMst
WHERE DID=1;
```

Thus, DML could be explained.

MAITRI K. HINGU

30. What is Data Control Language (DCL)?

"Who will access the data in database is managed by special language is called Data Control Language (DCL). DCL is set of command, used to Grant and Revoke permissions to and from the users."

Commands: GRANT……TO, REVOKE……FROM

1) GRANT……TO:

Usage: It is used to grant privileges(permission) to user.

Syntax:

```
GRANT objectPrivileges
ON objectName
TO username;
```

objectPrivileges: It can be ALTER, DELETE, INDEX, INSERT, SELECT, UPDATE.

objectName: It can be view name or table name.

username: User Id with which you are logged in.

Example:

```
GRANT SELECT, INSERT
ON DepartmentMst
TO abc_xyz;
```

MAITRI K. HINGU

2) REVOKE……FROM:

Usage: It is used to revoke (take back) privileges(permission) from user.

Syntax:

REVOKE objectPrivileges
ON objectName
FROM username;

objectPrivileges: It can be ALTER, DELETE, INDEX, INSERT, SELECT, UPDATE.
objectName: It can be view name or table name.
username: User Id with which you are logged in.

Example:

REVOKE SELECT, INSERT
ON DepartmentMst
FROM abc_xyz;

Thus, DCL could be explained.

MAITRI K. HINGU

31. What is Transaction Control Language (TCL)?

"Transactional Processing in a database is managed by special language is called Transactional Control Language (TCL). TCL is set of command, used to Save, Undo and Undo to the particular save point."

Commands: COMMIT, ROLLBACK, SAVEPOINT.

1) COMMIT:

Usage: It is used to save effect of your transaction in database.

Syntax:

```
COMMIT;
```

Example:

```
COMMIT;
```

2) SAVEPOINT:

Usage: It is used to create savepoint to which you can undo your actions later on by rollback.

Syntax:

```
SAVEPOINT savepointName;
```

Example:

```
SAVEPOINT sp1;
```

MAITRI K. HINGU

3) ROLLBACK:

Usage: It is used to undo the changes up to the last commit or up to the particular savepoint.

Syntax:

ROLLBACK [TO savepointName];

Example:

1) ROLLBACK;

2) ROLLBACK TO sp1;

It will undo all changes which has been done after the last commit.

ROLLBACK TO sp1;

It will undo all changes which has been done after the savepoint sp1 has been created.

Thus, TCL could be explained.

MAITRI K. HINGU

32. What is Super Key?

"A super key is a set of one or more columns that uniquely identifies any row within a Database Management System (DBMS) table."

Example: Super Key for EmployeeMst table is,

{EID, FName, MName, LName, Address, Contact, DOB}

Thus, Super Key could be defined.

33. What is Candidate Key?

"Candidate key is subset of Super key but only those subsets which are able to find record uniquely."

Example: Candidate key for EmployeeMst Table are,

{EID, FName, MName, LName}

{FName, MName, LName, Address, DOB}

{EID, FName, Address}

{EID, FName, Contact}

{EID, FName, DOB} ……

Thus, Candidate Key could be defined.

MAITRI K. HINGU

34. What is Minimal Candidate Key?

"Candidate key is subset of Super key but only those subsets which are able to find record uniquely as well as if we try to remove any column name from it then it will lose its meaning of finding records uniquely."

Example: Minimal Candidate key for EmployeeMst Table is,

{FName, MName, LName, Address, DOB}

Thus, Minimal Candidate Key could be defined.

35. What is Composite Key?

"A composite key is a combination of two or more columns in a table that can be used to uniquely identify each row in the table. Uniqueness is only guaranteed when the columns are combined; when taken individually the columns do not guarantee uniqueness."

Thus, Composite Key could be defined.

36. What is Alternate Key?

"An alternate key is a key associated with one or more columns whose values uniquely identify every row in the table, but which is not the primary key."

Out of all candidate keys, only one gets selected as primary key, remaining keys are known as alternative or secondary keys.

MAITRI K. HINGU

For example:

Alternate Key {FName, MName, LName, DOB, Address}

Thus, Alternate Key could be defined

37. What is Functional Dependency?

"In relational database, a functional dependency is a constraint between two sets of attributes in a relation(Table) from a database. "

In other words, functional dependency is a constraint that describes the relationship between attributes in a relation.

Given a relation R, a set of attributes X in R is said to functionally determine another set of attributes Y, also in R, (written X → Y) if and only if, each X value in R is associated with precisely one Y value in R; R is then said to satisfy the functional dependency X → Y.

In other words, a dependency FD: X → Y means that the values of Y are determined by the values of X. Two tuples sharing the same values of X will necessarily have the same values of Y.

X: {EID}

Y: {FName}

{EID} → {FName}

In above example, we can say that FName is functionally dependent on EID.

Thus, Functional Dependency could be explained.

MAITRI K. HINGU

38. What is Fully Functional Dependency?

"Functional dependency FD: X → Y is said to be fully functional dependency if removal of any attribute from X will not hold any functional dependency between X & Y."

X: {FName, MName, LName, Address, DOB}

Y: {DID}

With the above set of attribute, we can say that Y is Fully Functionally dependent on X, because removal any of attribute from X will no longer find value of Y uniquely.

Note: Whenever X is a Minimal Candidate Key, we can say that Y is fully functionally dependent on X.

Thus, Fully Functional Dependency could be defined.

39. What is Partial Functional Dependency?

"Functional dependency, FD: X → Y is said to be partial functional dependency, in which Y is non-prime attribute and X is part of candidate key but not the whole candidate key so, we can say that Y is partially dependent on X."

EmployeeMst (EID, FName, MName, LName, Address, DID)

P.K. = EID

Candidate Key = EID, FName

So, Partial functional dependency will be,

FName → MName, FName → LName

MAITRI K. HINGU

FName → Address, FName → DID

Thus, Partial Functional Dependency could be defined.

40. What is Trivial Functional Dependency?

"Functional dependency, FD: X → Y is said to be trivial functional dependency, in which Y is subset of X or we can say that RHS is a subset of LHS."

L.H.S. = X: {EID, FName, MName, LName}
R.H.S. = Y: {FName, LName}

{EID, FName, MName, LName} → {FName, LName}

Thus, Trivial Functional Dependency could be defined.

41. What is Non-Trivial Functional Dependency?

There are two types of non-trivial functional dependency.

1) Complete Non-Trivial Functional Dependency:

A Functional Dependency is completely non-trivial if none of the RHS attributes are part of the LHS attributes.

L.H.S. = X: {EID, FName}
R.H.S. = Y: {MName, LName}

{EID, FName} → {MName, LName}

MAITRI K. HINGU

2) Semi Non-Trivial Functional Dependency:

A Functional Dependency is semi non-trivial if at least one of the RHS attributes are not part of the LHS attributes.

L.H.S. = X: {EID, FName}

R.H.S. = Y: {FName, LName, DID}

{EID, FName} → {FName, LName, DID}

Thus, Non-Trivial Functional Dependency could be explained.

MAITRI K. HINGU

42. What is Multivalued Dependency?

"In contrast to the functional dependency, the multivalued dependency requires that certain tuples be present in a relation. Therefore, a multivalued dependency is a special case of tuple-generating dependency."

Multivalued Dependency is denoted by $\alpha \rightarrow\rightarrow \beta$.

Let R be the relational schema and $\alpha \subseteq R$ and $\beta \subseteq R$. The multivalued dependency $\alpha \rightarrow\rightarrow \beta$ (α multi determines β) holds on R if for all pairs of tuple t1, t2, t3 and t4 following is true.

R: EID, Name, Address, ProjectID

α: EID, FName

β: Address

$R - \beta - \alpha$: ProjectID

$t1[\alpha] = t2[\alpha] = t3[\alpha] = t4[\alpha]$

$t3[\beta] = t1[\beta]$

$t3[R - \beta - \alpha] = t2[R - \beta - \alpha]$

$t4[\beta] = t2[\beta]$

$t4[R - \beta - \alpha] = t1[R - \beta - \alpha]$

MAITRI K. HINGU

Example:

	EID	Name	Address	ProjectID
T1	E01	Haresh Patel	Res Course Road	P01
T2	E01	Haresh Patel	M. G. Road	P02
T3	E01	Haresh Patel	Res Course Road	P02
T4	E01	Haresh Patel	M. G. Road	P01

t1[EID, Name] = t2[EID, Name] = t3[EID, Name] = t4[EID, Name]

t3[Address] = t1[Address]

t3[ProjectID] = t2[ProjectID]

t4[Address] = t2[Address]

t4[ProjectID] = t1[ProjectID]

Thus, Multivalued Dependency could be explained.

MAITRI K. HINGU

43. What is Transitive Functional Dependency?

"Functional dependency, FD: X → Z is said to be Transitive functional dependency, in which Z is indirectly dependent on X, which means that X → Y and Y → Z"

X: {EID}

Y: {DID}

Z: {DName}

So, {EID} → {DID} and {DID} → {DName} so we can say that {EID} → {DName}

Thus, Transitive Functional Dependency could be explained.

44. What are Armstrong's Axioms? Explain it in detail.

OR

What is Closure of functional Dependency? Explain it in detail.

- Armstrong's Axioms is a set of rules.
- It provides a simple technique for reasoning about functional dependencies.
- It was developed by William W. Armstrong in 1974.
- It is used to infer all the functional dependencies on a relational database.

MAITRI K. HINGU

Various Armstrong's Rules

A. Primary Rules

Rule 1	**Reflexivity Rule:** If A is a set of attributes and B is a subset of A, then A holds B. {A → B} A: {EID, FName, LName} B: {FName, LName} {EID, FName, LName} → {FName, LName}
Rule 2	**Augmentation Rule:** If A hold B and C is a set of attributes, then AC holds BC. {AC → BC} It means that attribute in dependencies does not change the basic dependencies. A: {EID} B: {FName} C: {DOB} {EID, DOB} → {FName, DOB}
Rule 3	**Transitivity Rule:** If A holds B and B holds C, then A holds C. If {A → B} and {B → C}, then {A → C} A holds B {A → B} means that A functionally determines B. A: {EID} B: {DID} C: {DName} {EID} → {DID}, {DID} → {DName} So, we can say that, {EID} → {DName}

MAITRI K. HINGU

B. Secondary Rules

Rule 1	**Union Rule:** If A holds B and A holds C, then A holds BC. If {A → B} and {A → C}, then {A → BC} A: {EID} B: {FName} C: {LName} {EID} → {FName}, {EID} → {LName} So we can say that, {EID} → {FName, LName}
Rule 2	**Decomposition Rule:** If A holds BC and A holds B, then A holds C. If {A → BC} and {A → B}, then {A → C} A: {EID} B: {FName} C: {LName} {EID} → {FName, LName} So we can say that, {EID} → {FName} as well as {EID} → {LName}

MAITRI K. HINGU

Rule 3	**Pseudo Transitivity Rule:** If A holds B and BC holds D, then AC holds D. If {A → B} and {BC → D}, then {AC → D} A: {EID} B: {DOB} C: {FName} D: {DID} {EID} → {DOB}, {DOB, FName} → {DID} So, we can say that, {EID, FName} → {DID}

Thus, Closure of Functional Dependency could be explained.

Sometimes Functional Dependency Sets are not able to reduce if the set has following properties,

- The Right-hand side set of functional dependency holds only one attribute.
- The Left-hand side set of functional dependency cannot be reduced, it changes the entire content of the set.
- Reducing any functional dependency may change the content of the set.

A set of functional dependencies with the above three properties are also called as Canonical or Minimal Functional Dependency.

MAITRI K. HINGU

Example:
Consider relation E = (P, Q, R, S, T, U) having set of Functional Dependencies (FD).

P → Q P → R
QR → S Q → T
QR → U PR → U

Derived following with the help of Axioms: (Closure of Functional Dependency)

1. P → T
2. PR → S
3. QR → SU
4. PR → SU

Solution:

1. P → T

In the above FD set, P → Q and Q → T
So, Using Transitive Rule: If {A → B} and {B → C}, then {A → C}
∴ If P → Q and Q → T, then P → T.
P → T

2. PR → S

In the above FD set, P → Q
As, QR → S
So, Using Pseudo Transitivity Rule: If {A → B} and {BC → D}, then {AC → D}

MAITRI K. HINGU

∴ If P → Q and QR → S, then PR → S.

PR → S

3. **QR → SU**

In above FD set, QR → S and QR → U

So, Using Union Rule: If {A → B} and {A → C}, then {A → BC}

∴ If QR → S and QR → U, then QR → SU.

QR → SU

4. **PR → SU**

So, Using Pseudo Transitivity Rule: If {A → B} and {BC → D}, then {AC → D}

∴ If PR → S and PR → U, then PR → SU.

PR → SU

MAITRI K. HINGU

45. What is Generalization?

"Depending on common attributes of lower level entity, we are able to create another higher-level entity. This process is known as generalization."

Thus, Generalization could be explained.

46. What is Specialization?

"The process of sub-grouping depending on specific attributes at the lower level is known as specialization."

Thus, Specialization could be explained.

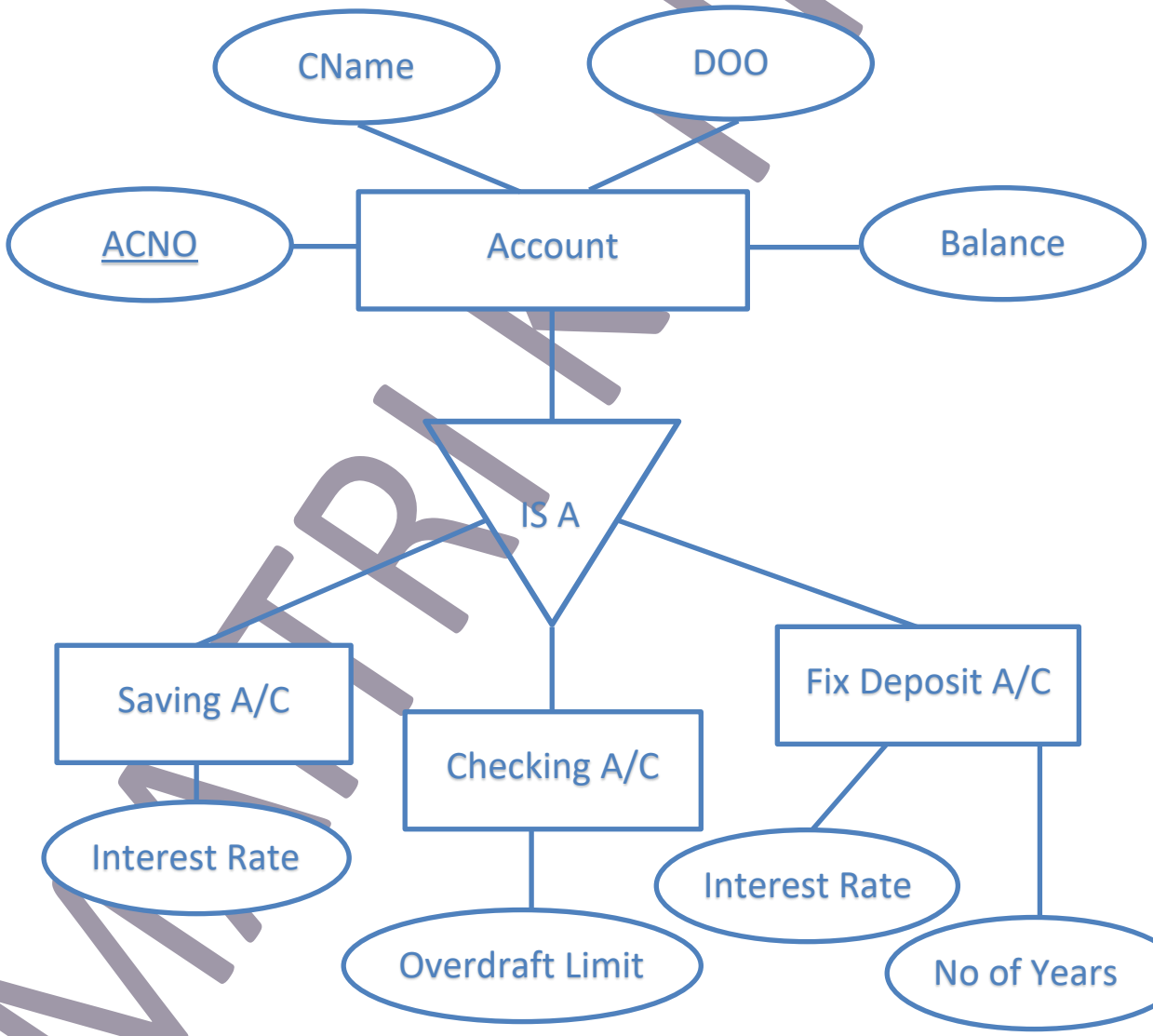
MAITRI K. HINGU

47. What is "IS A" Relationship?

"IS A" relationship exist between super type and sub type entity, which is used in generalization and specialization.

"IS A" relationship is represented by inverted triangle between entities.

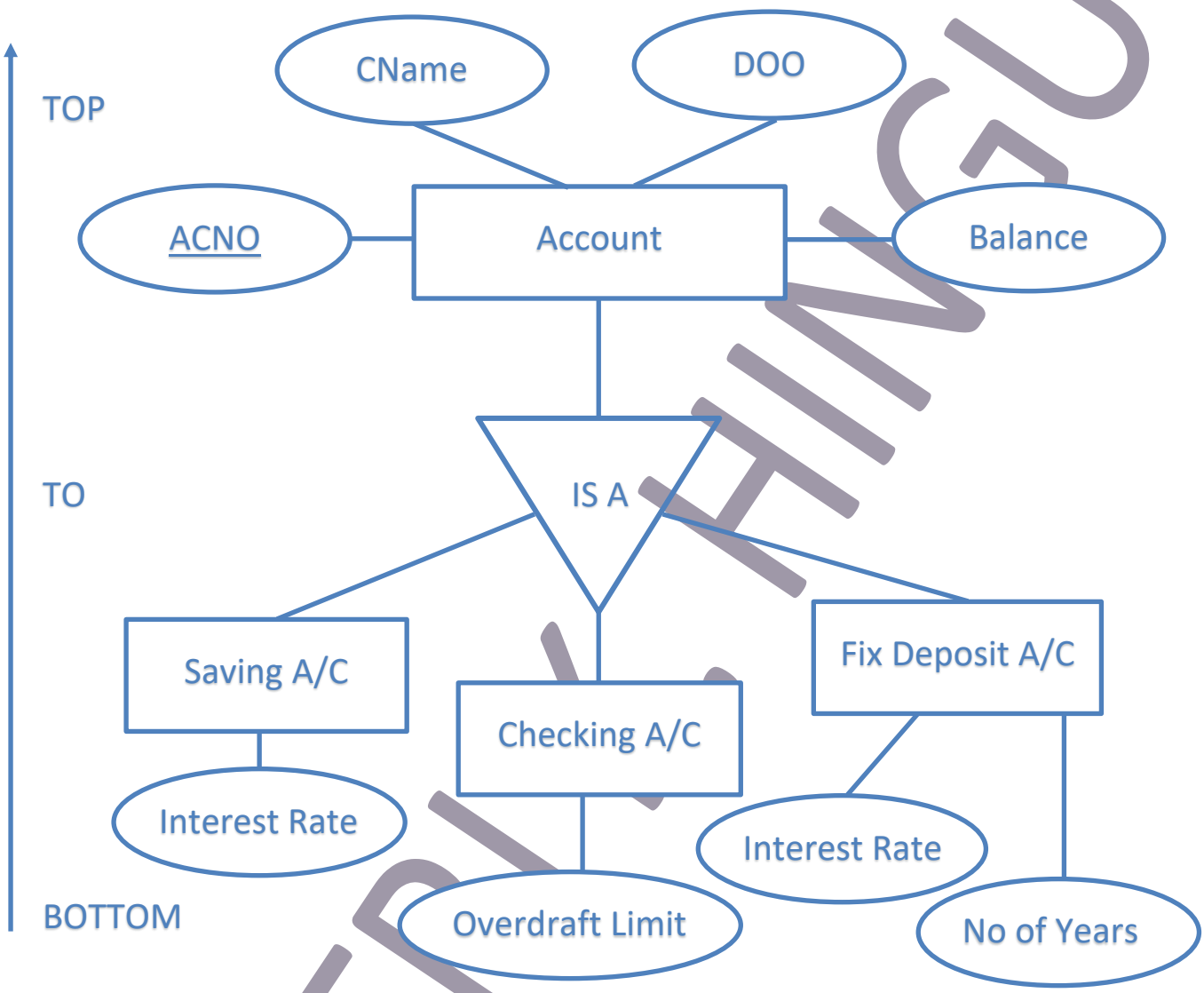
In below example, we can say that Account "IS A" Saving account or Checking account or Fix deposit account.



Thus, "IS A" relationship could be explained

MAITRI HINGU

48. Explain Generalization on Account Entity.



Generalization:

"Depending on common attributes of lower level entities, we are able to create one higher-level entity. This process is known as generalization."

The process of generalization goes from bottom to top manner.

MAITRI HINGU

Attributes on whose dependence we can create one higher level entity set, they are in sharable mode and we can call them common attributes.

In this account example, we have multiple entities at the lower level that are,

i) Saving Account which has ACNO, Balance, DOO (Date Of Opening), CName and Interest Rate attributes.

ii) Checking Account or Current Account which has ACNO, Balance, DOO (Date Of Opening), CName and Overdraft Limit attributes.

iii) Fix Deposit Account ACNO, Balance, DOO (Date Of Opening), CName, Interest Rate and No of Years attributes.

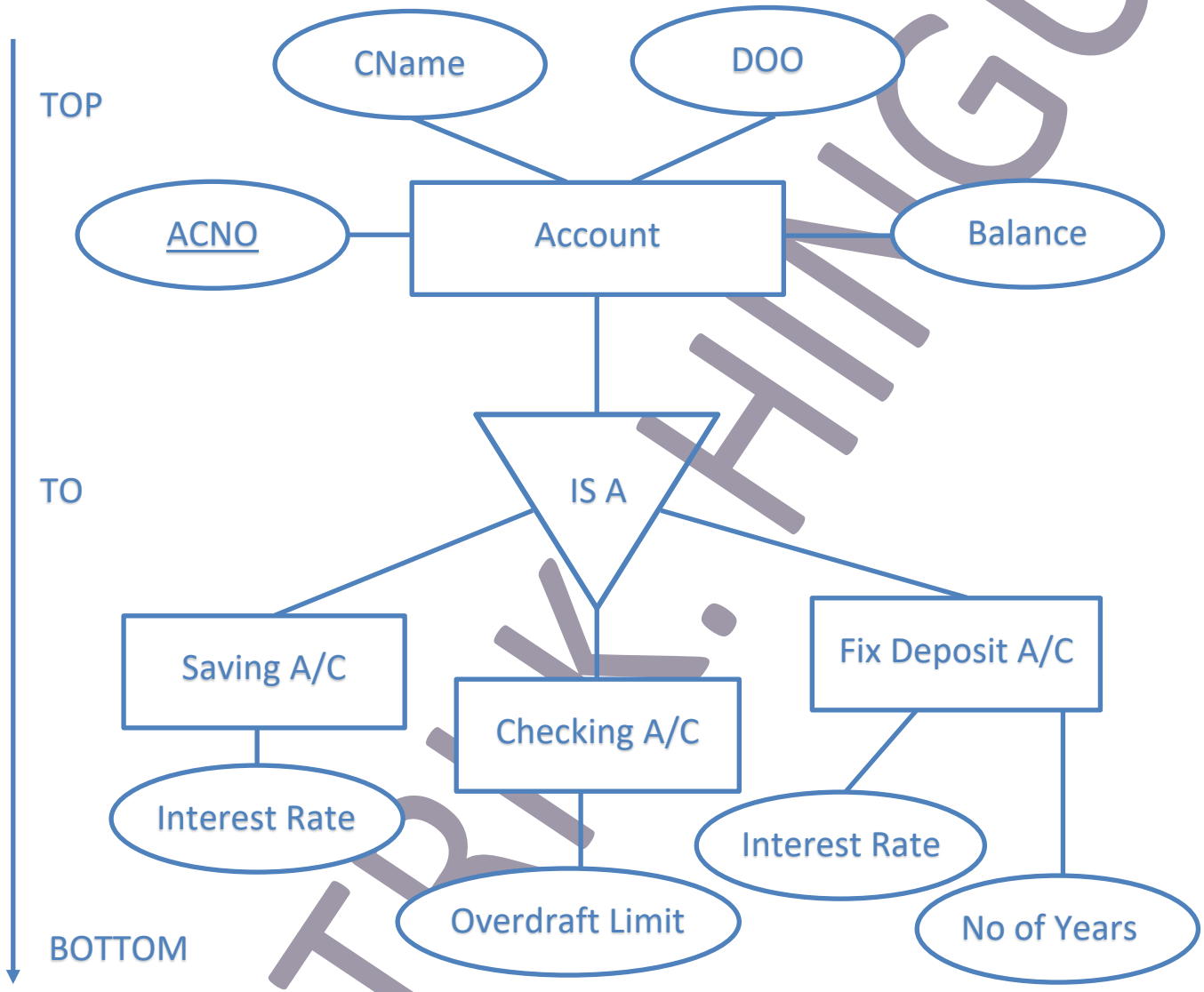
Depending on common attributes which are ACNO, Balance, DOO (Date Of Opening) and CName, we are creating another entity at higher level namely, Account.

Account Entity will hold all common attributes of lower level entity.

Thus, Generalization on Account entity could be Explained.

MAITRI K. HINGU

49. Explain Specialization on Account Entity.



Specialization:

"The process of creating sub-group or sub entity depending on specific attribute at the lower level known as specialization."

MAITRI K. HINGU

The process of specialization goes from top to bottom manner.

Attributes on whose dependence we can create multiple entity set at lower level, they cannot be shared and we can call them special or specific attribute of entity.

In this account example, at higher level account has ACNO, Balance, DOO (Date Of Opening), CName attributes.

We are creating multiple entities depending on specific attributes that are,

i) Interest Rate
ii) Overdraft Limit
iii) No of Years

Depending on these three distinct attributes which are Interest Rate, Overdraft Limit and No of years, we are creating another three entity at lower level are,

i) Saving Account: It will contain Interest rate attribute as well as all common attributes of Account entity.
ii) Checking Account: It will contain Overdraft limit attribute as well as all common attributes of Account entity.
iii) Fix Deposit Account: It will contain Interest rate, No of years as well as all common attributes of Account entity.

MAITRI K. HINGU

Here, Interest Rate attribute is being hold by Saving Account as well as Fix Deposit Account which cannot be hold by Checking Account.

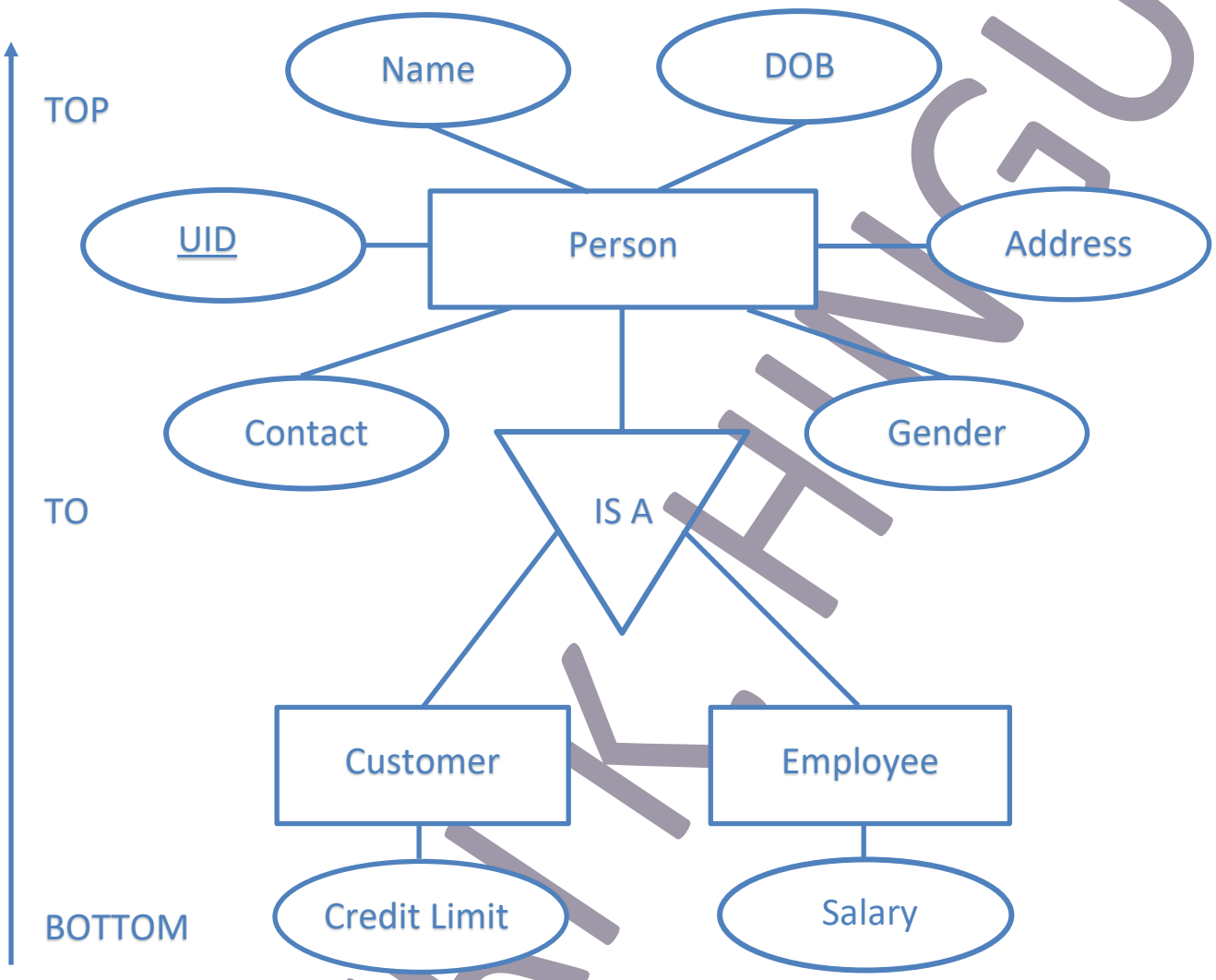
Overdraft Limit attribute is being hold by Checking Account which cannot be hold by Saving Account as well as Fix Deposit Account.

No of Years attribute is being hold by Fix Deposit Account which cannot be hold by Saving Account as well as Checking Account.

Thus, Specialization on Account entity could be Explained.

MAITRI K. HINGU

50. Explain Generalization on Person Entity.



Generalization:

"Depending on common attributes of lower level entities, we are able to create one higher-level entity. This process is known as generalization."

The process of generalization goes from bottom to top manner.

MAITRI HINGU

Attributes on whose dependence we can create one higher level entity set, they are in sharable mode and we can call them common attributes.

In this Person example, we have multiple entities at the lower level that are,

i) Customer which has UID, Name, DOB (Date Of Birth), Gender, Address, Contact and Credit Limit attributes.

ii) Employee which has UID, Name, DOB (Date Of Birth), Gender, Address, Contact and Salary attributes.

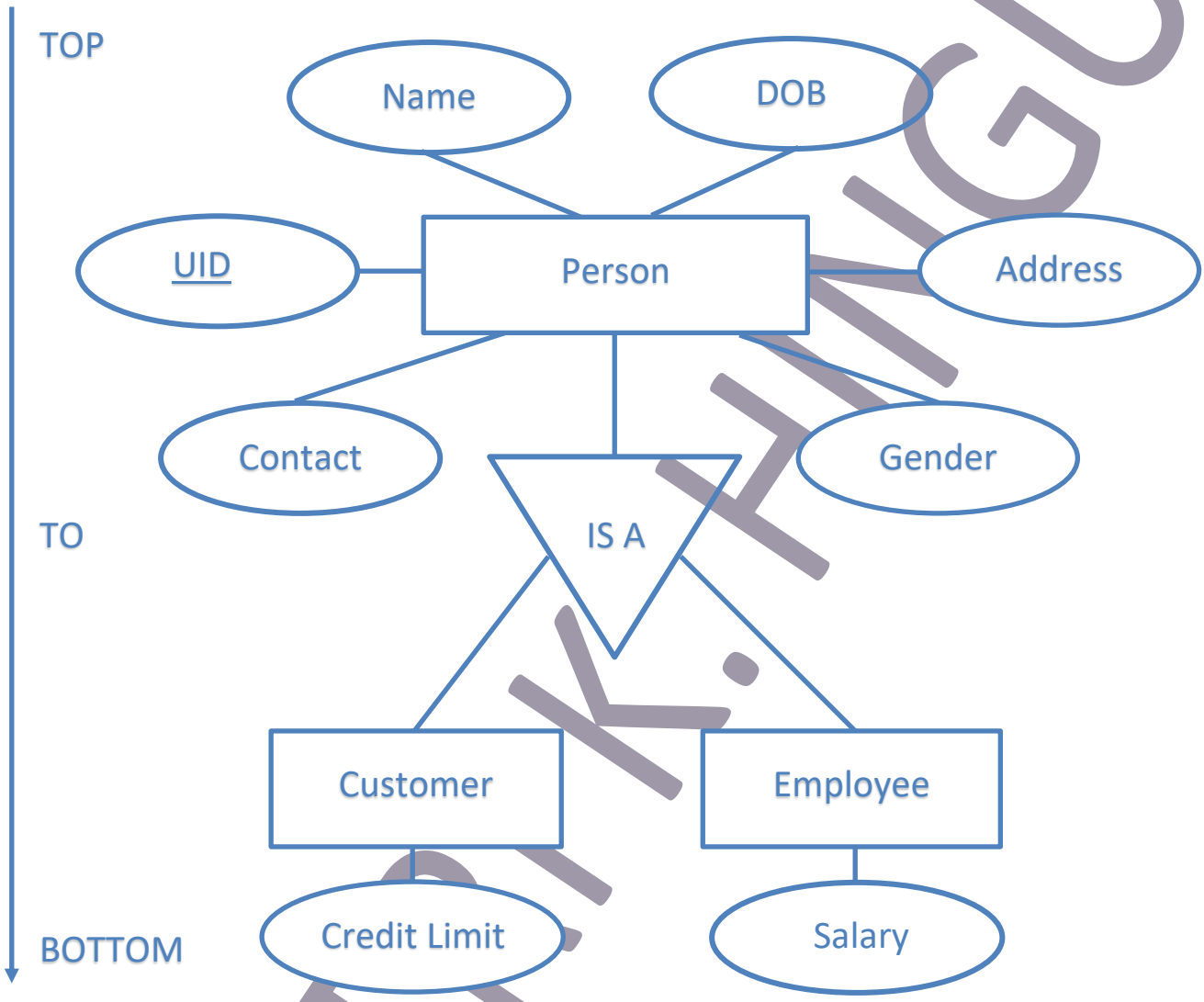
Depending on common attributes which are UID, Name, DOB (Date Of Birth), Gender, Address and Contact, we are creating another entity at higher level namely, Person.

Person Entity will hold all common attributes of lower level entity.

Thus, Generalization on Person entity could be Explained.

MAITRI K. HINGU

51. Explain Specialization on Person Entity.



Specialization:

"The process of creating sub-group or sub entity depending on specific attribute at the lower level known as specialization."

The process of specialization goes from top to bottom manner.

MAITRI K. HINGU

Attributes on whose dependence we can create multiple entity set at lower level, they cannot be shared and we can call them special or specific attribute of entity.

In this Person example, at higher level Person has UID, Name, DOB (Date Of Birth), Address, Contact and Gender attributes.

We are creating multiple entities depending on specific attributes that are,

i) Credit Limit

ii) Salary

Depending on these two distinct attributes which are Credit Limit and Salary, we are creating another two entity at lower level are,

i) Customer: It will contain Credit Limit attribute as well as all common attributes of Person entity.

ii) Employee: It will contain Salary attribute as well as all common attributes of Person entity.

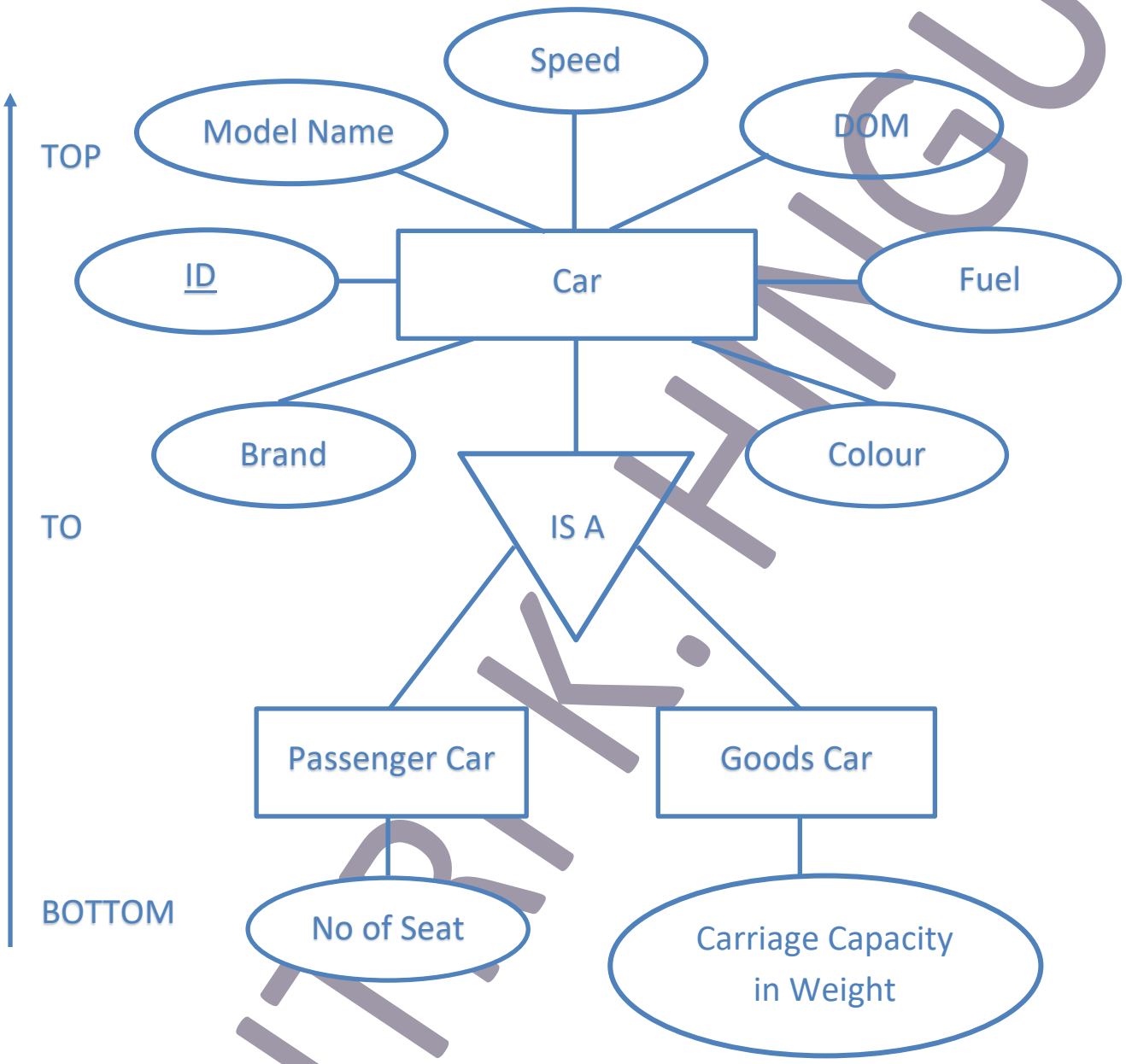
Here, Credit Limit attribute is being hold by Customer which cannot be hold by Employee.

Salary attribute is being hold by Employee which cannot be hold by Customer.

Thus, Specialization on Person entity could be Explained.

MAITRI K. HINGU

52. Explain Generalization on Car Entity.



Generalization:

"Depending on common attributes of lower level entities, we are able to create one higher-level entity. This process is known as generalization."

MAITRI K. HINGU

The process of generalization goes from bottom to top manner.

Attributes on whose dependence we can create one higher level entity set, they are in sharable mode and we can call them common attributes.

In this Car example, we have multiple entities at the lower level that are,

i) Passenger Car which has ID, Model Name, DOM (Date Of Manufacturing), Brand, Fuel, Colour, Speed and No of Seat attributes.

ii) Goods Car which has ID, Model Name, DOM (Date Of Manufacturing), Brand, Fuel, Colour, Speed and Carriage Capacity in Weight attributes.

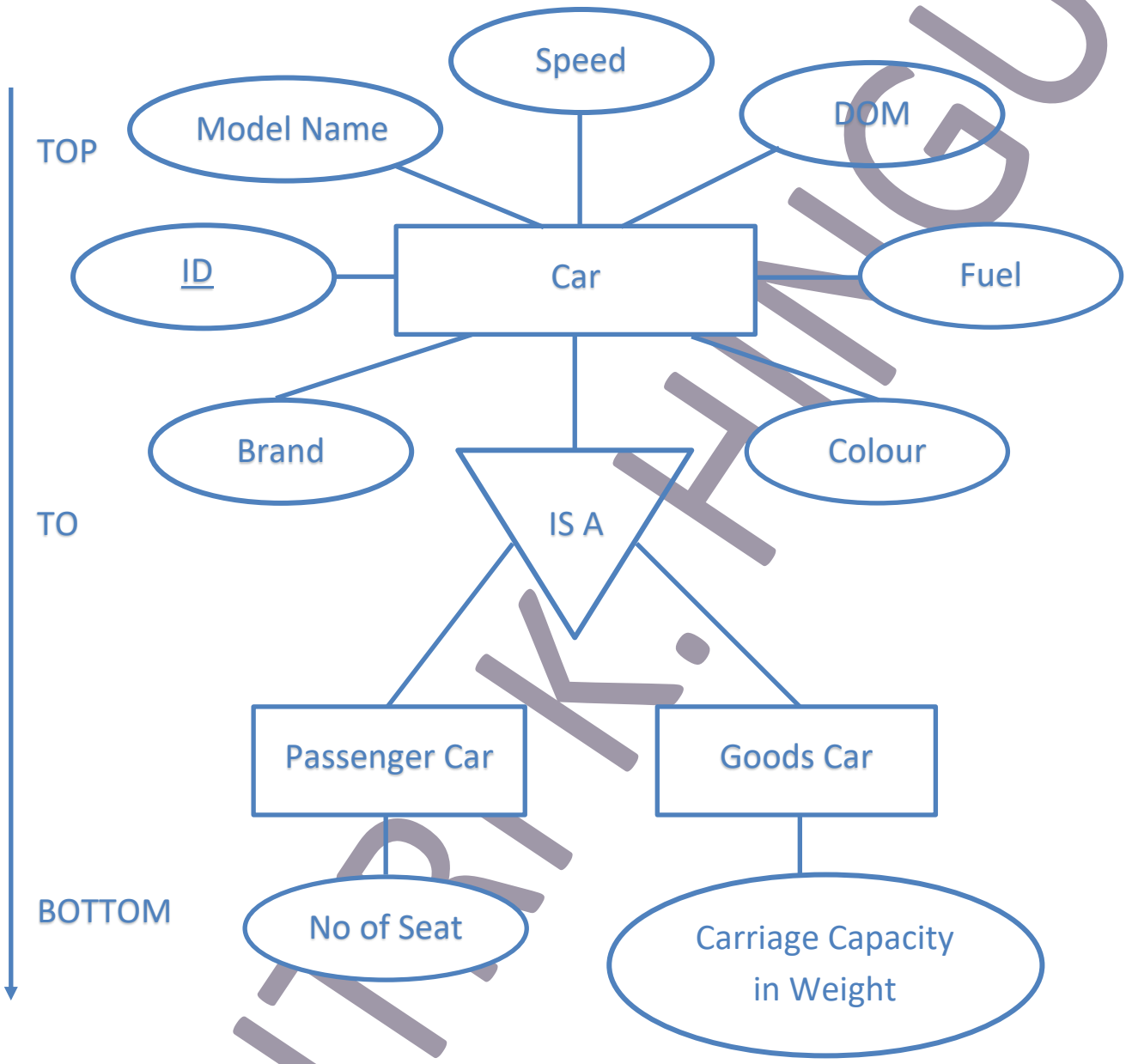
Depending on common attributes which are ID, Model Name, DOM (Date Of Manufacturing), Brand, Fuel, Colour and Speed, we are creating another entity at higher level namely, Car.

Car Entity will hold all common attributes of lower level entity.

Thus, Generalization on Car entity could be Explained.

MAITRI K. HINGU

53. Explain Specialization on Car Entity.



Specialization:

"The process of creating sub-group or sub entity depending on specific attribute at the lower level known as specialization."

MAITRI K. HINGU

The process of specialization goes from top to bottom manner.

Attributes on whose dependence we can create multiple entity set at lower level, they cannot be shared and we can call them special or specific attribute of entity.

In this Car example, at higher level Car has ID, Model Name, DOM (Date Of Manufacturing), Brand, Fuel, Colour and Speed attributes.

We are creating multiple entities depending on specific attributes that are,

i) No of Seat

ii) Carriage Capacity in Weight

Depending on these two distinct attributes which are No of Seat and Carriage Capacity in Weight, we are creating another two entity at lower level are,

i) Passenger Car: It will contain No of Seat attribute as well as all common attributes of Car entity.

ii) Goods Car: It will contain Carriage Capacity in Weight attribute as well as all common attributes of Car entity.

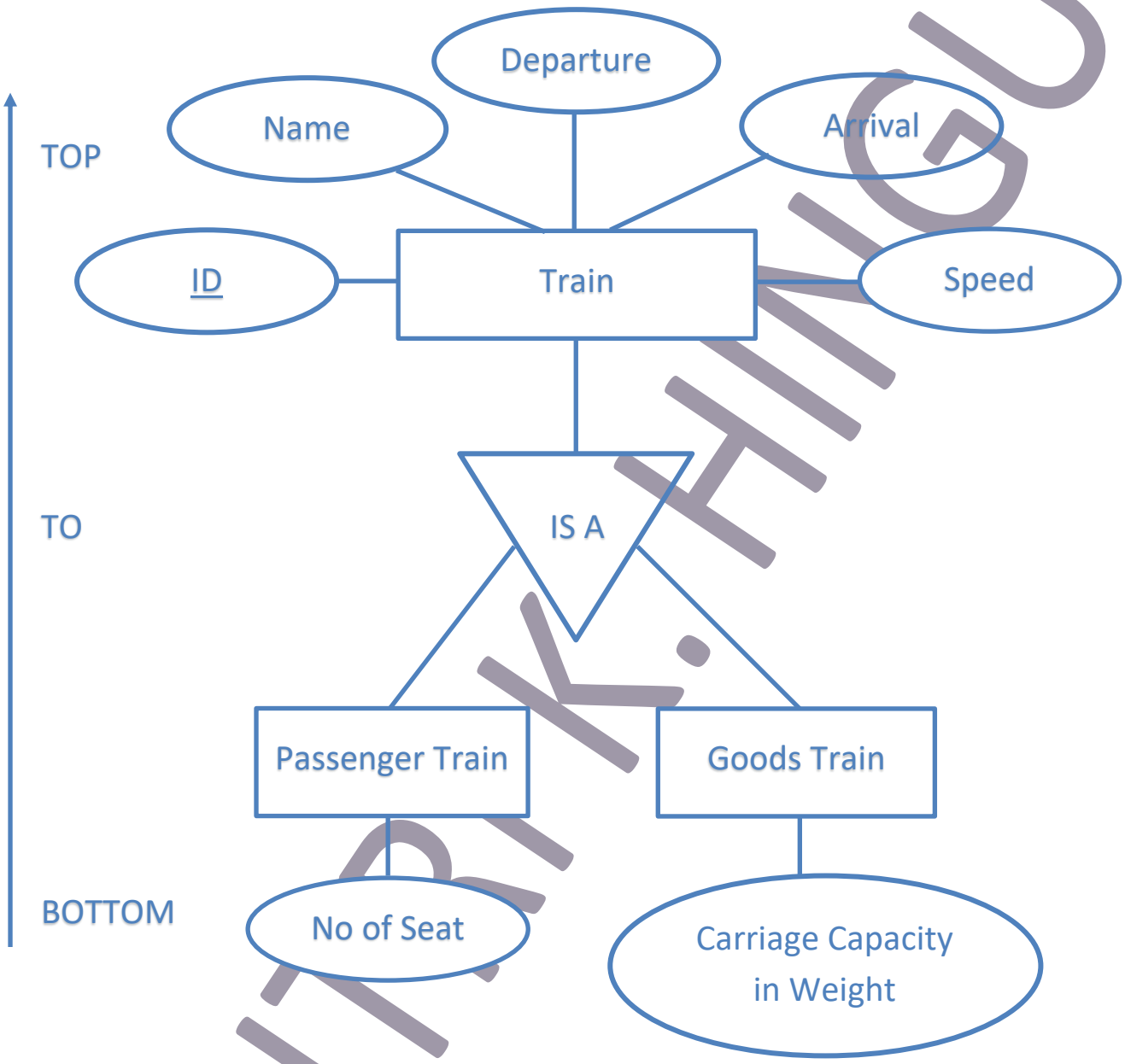
Here, No of Seat attribute is being hold by Passenger Car which cannot be hold by Goods Car.

Carriage Capacity in Weight attribute is being hold by Goods Car which cannot be hold by Passenger Car.

Thus, Specialization on Car entity could be Explained.

MAITRI K. HINGU

54. Explain Generalization on Train Entity.



Generalization:

"Depending on common attributes of lower level entities, we are able to create one higher-level entity. This process is known as generalization."

MAITRI K. HINGU

The process of generalization goes from bottom to top manner.

Attributes on whose dependence we can create one higher level entity set, they are in sharable mode and we can call them common attributes.

In this Train example, we have multiple entities at the lower level that are,

i) Passenger Train which has ID, Name, Speed, Arrival, Departure and No of Seat attributes.

ii) Goods Train which has ID, Name, Speed, Arrival, Departure and Carriage Capacity in Weight attributes.

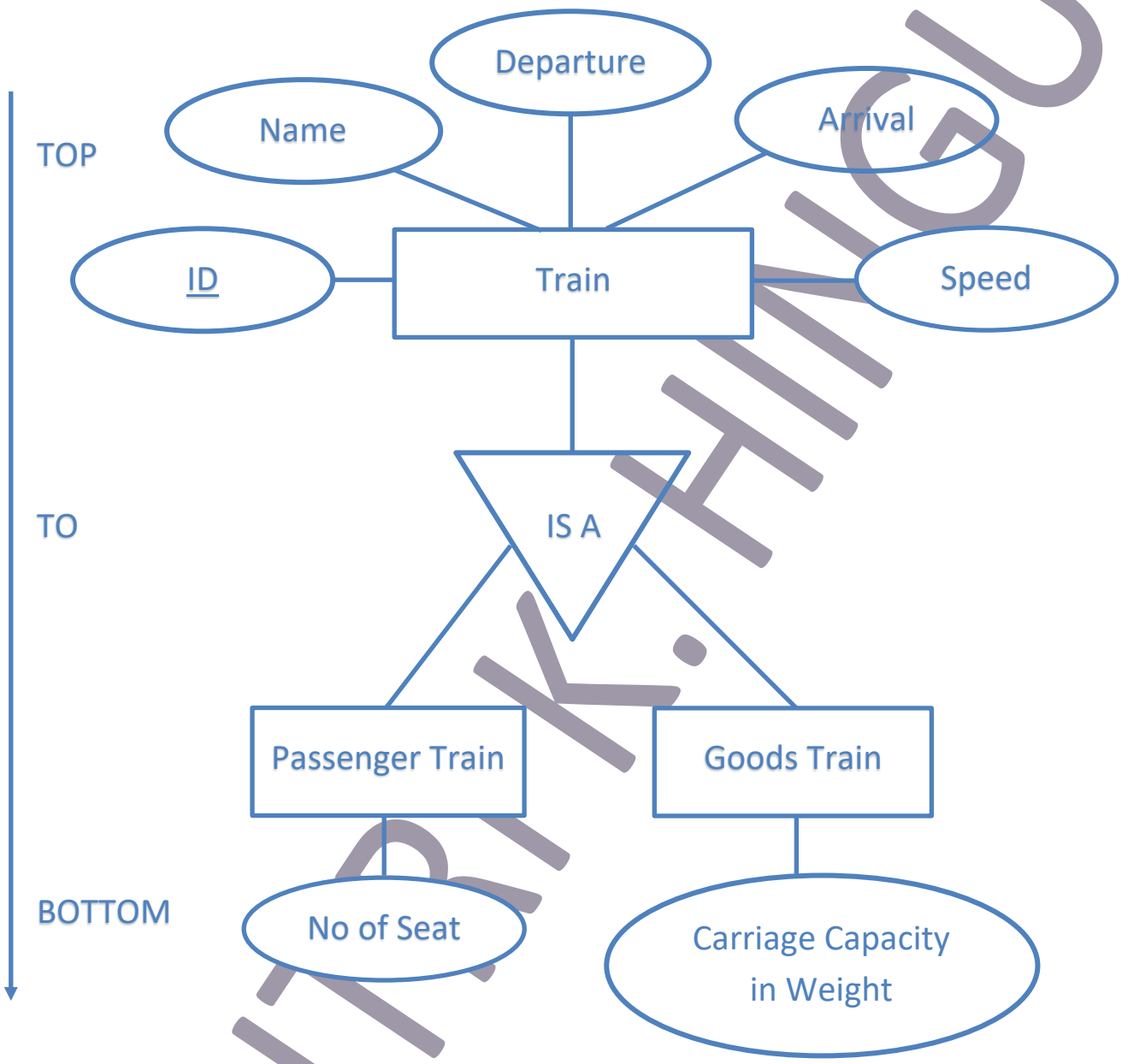
Depending on common attributes which are ID, Name, Speed, Arrival and Departure, we are creating another entity at higher level namely, Train.

Train Entity will hold all common attributes of lower level entity.

Thus, Generalization on Train entity could be Explained.

MAITRI K. HINGU

55. Explain Specialization on Train Entity.



Specialization:

"The process of creating sub-group or sub entity depending on specific attribute at the lower level known as specialization."

MAITRI K. HINGU

The process of specialization goes from top to bottom manner.

Attributes on whose dependence we can create multiple entity set at lower level, they cannot be shared and we can call them special or specific attribute of entity.

In this Train example, at higher level Train has ID, Name, Departure, Arrival and Speed attributes.

We are creating multiple entities depending on specific attributes that are,

i) No of Seat
ii) Carriage Capacity in Weight

Depending on these two distinct attributes which are No of Seat and Carriage Capacity in Weight, we are creating another two entity at lower level are,

i) Passenger Train: It will contain No of Seat attribute as well as all common attributes of Car entity.
ii) Goods Train: It will contain Carriage Capacity in Weight attribute as well as all common attributes of Car entity.

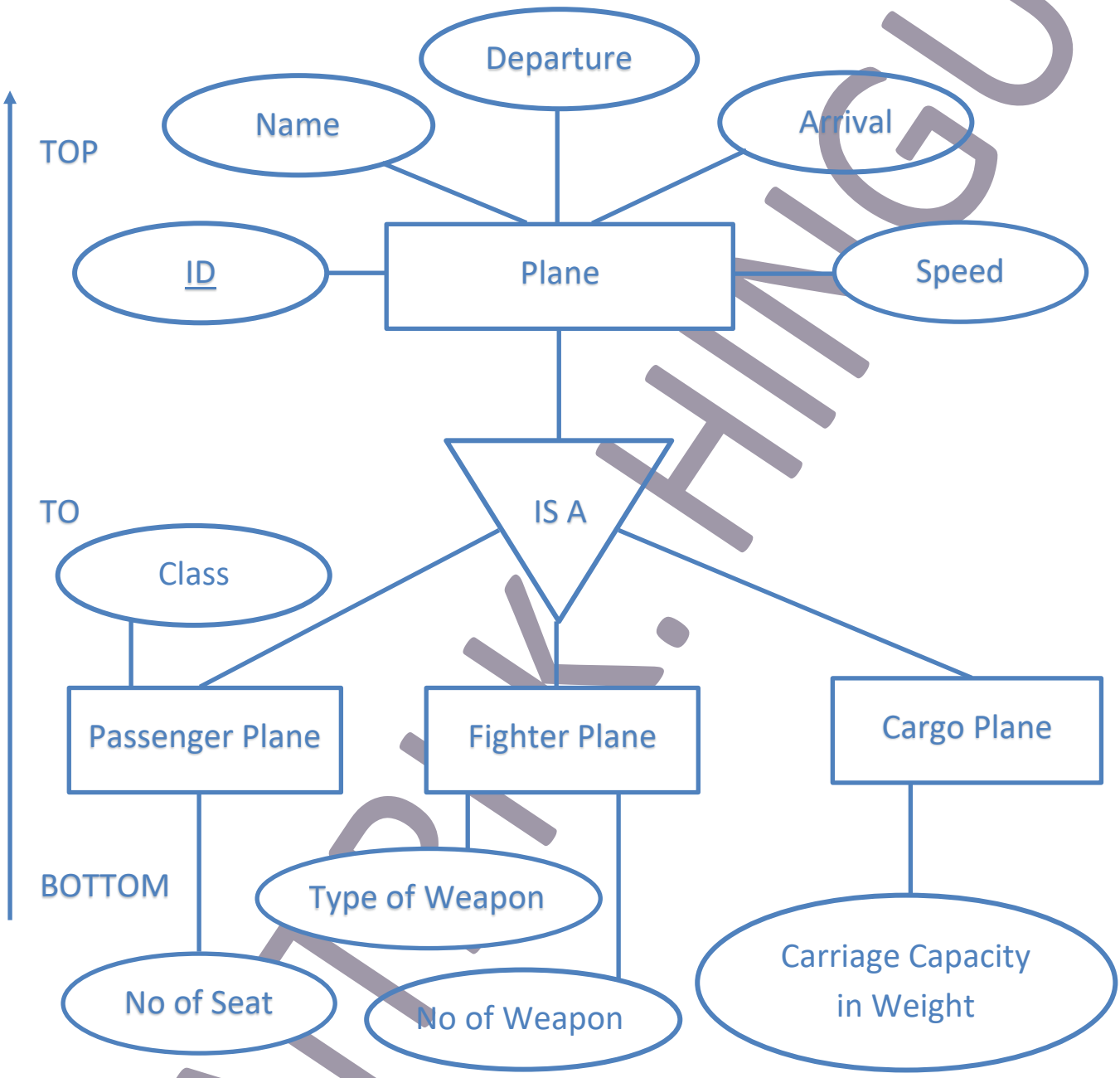
Here, No of Seat attribute is being hold by Passenger Train which cannot be hold by Goods Train.

Carriage Capacity in Weight attribute is being hold by Goods Train which cannot be hold by Passenger Train.

Thus, Specialization on Train entity could be Explained.

MAITRI K. HINGU

56. Explain Generalization on Plane Entity.



Generalization:

"Depending on common attributes of lower level entities, we are able to create one higher-level entity. This process is known as generalization."

MAITRI HINGU

The process of generalization goes from bottom to top manner.

Attributes on whose dependence we can create one higher level entity set, they are in sharable mode and we can call them common attributes or general attributes.

In this Plane example, we have multiple entities at the lower level that are,

i) Passenger Plane which has ID, Name, Speed, Arrival, Departure, Class and No of Seat attributes.

ii) Fighter Plane which has ID, Name, Speed, Arrival, Departure, Type of Weapon and No of weapon attributes.

iii) Cargo Plane which has ID, Name, Speed, Arrival, Departure and Carriage Capacity in Weight attributes.

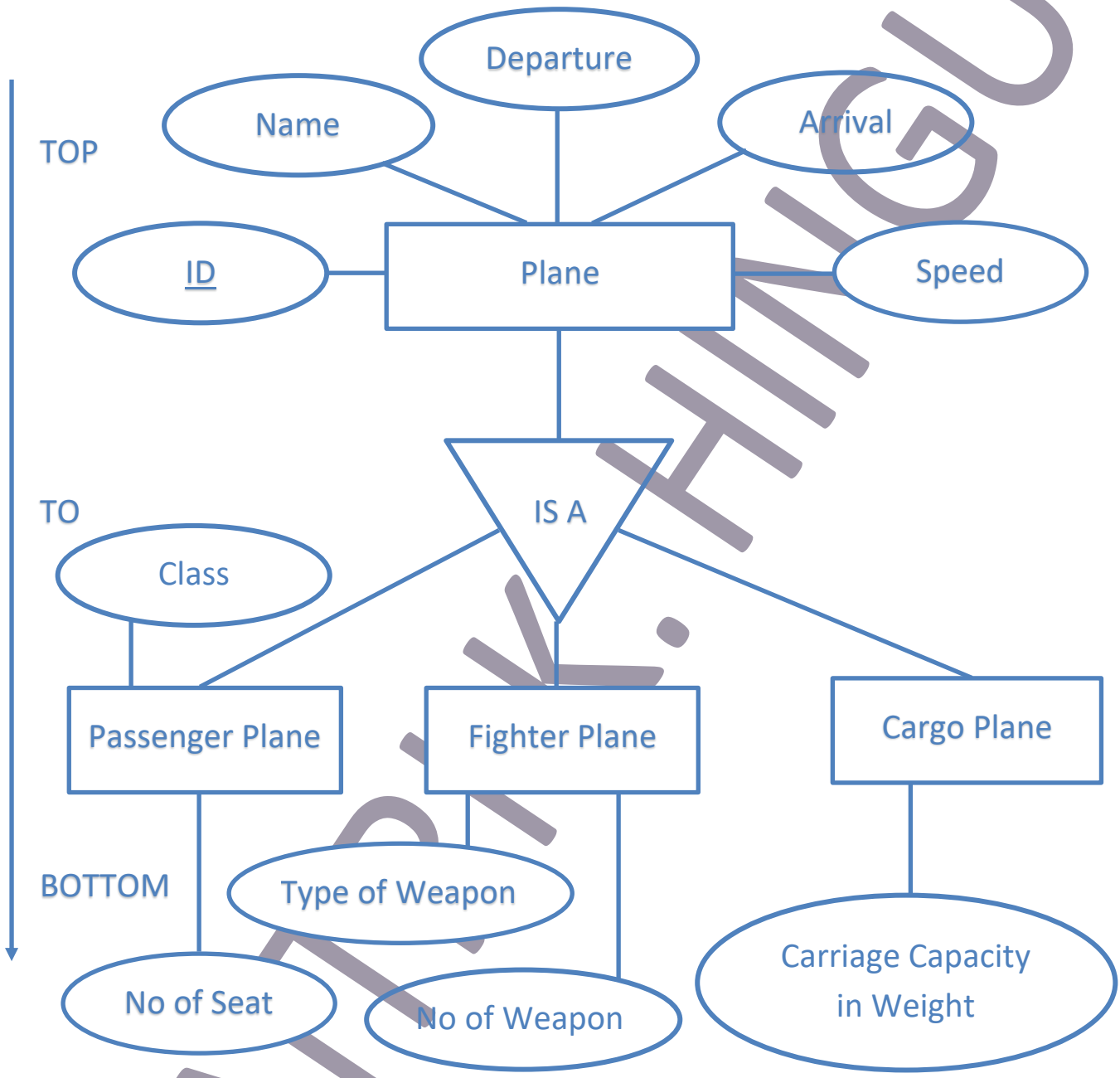
Depending on common attributes which are ID, Name, Speed, Arrival and Departure, we are creating another entity at higher level namely, Plane.

Plane Entity will hold all common attributes of lower level entity.

Thus, Generalization on Plane entity could be Explained.

MAITRI K. HINGU

57. Explain Specialization on Plane Entity.



Specialization:

"The process of creating sub-group or sub entity depending on specific attribute at the lower level known as specialization."

MAITRI HINGU

The process of specialization goes from top to bottom manner.

Attributes on whose dependence we can create multiple entity set at lower level, they cannot be shared and we can call them special or specific attribute of entity.

In this Plane example, at higher level Train has ID, Name, Departure, Arrival and Speed attributes.

We are creating multiple entities depending on specific attributes that are,

i) No of Seat
ii) Class
iii) Type of Weapon
iv) No of Weapon
v) Carriage Capacity in Weight

Depending on these five distinct attributes which are No of Seat, Class, Type of Weapon, No of Weapon and Carriage Capacity in Weight, we are creating another three entity at lower level are,

i) Passenger Plane: It will contain No of Seat and Class attribute as well as all common attributes of Car entity.
ii) Fighter Plane: It will contain Type of Weapon and No of Weapon attribute as well as all common attributes of Car entity.
iii) Cargo Plane: It will contain Carriage Capacity in Weight attribute as well as all common attributes of Car entity.

MAITRI K. HINGU

Here, No of Seat and Class attribute is being hold by Passenger Plane which cannot be hold by Fighter Plane and Cargo Plane.

Type of Weapon and No of Weapon attribute is being hold by Fighter Plane which cannot be hold by Passenger Plane and Cargo Plane.

Carriage Capacity in Weight attribute is being hold by Cargo Plane which cannot be hold by Passenger Plane and Fighter Plane.

Thus, Specialization on Plane entity could be Explained.

MAITRI K. HINGU

58. What are Anomalies? Explain its types in detail.

OR

Explain Need of Normalization.

"Anomalies are the problem or error which occurred in poorly construct and unnormalized database where all data are stored in one table (flat file database)."

Consider following flat table for example.

EID	Name	Address	Project	DOJ	DName
E01	Haresh Patel	Surat	P01	1-1-2008	HR
E02	Disha Shah	Baroda	P01	4-2-2010	Export
E02	Disha Shah	Baroda	P01	4-2-2010	Finance
E03	Nisha Sharma	Mumbai	P02	6-7-2011	Finance
E03	Nisha Sharma	Mumbai	P02	6-7-2011	HR

There are three types of anomaly which are,

1) **Insertion Anomaly:**

Insertion anomaly occurs when we are unable to add data to the database due to absence of other the data.

For Example:

Company is declaring the new department that is 'Production' but selection of employees is remaining. If we want to insert department name 'Production' that will not be possible because of absence of employee data. (all fields are created with not null constraint)

MAITRI K. HINGU

2) Update Anomalies:

Update anomaly occurs when we are unable to change all data in the database due to redundancy of the data which leads to data inconsistency.

For Example:

Employee Disha Shah has been shifted from Finance to Import and she will not continue work in Finance department then which bases we will update the record? And if we update on the bases of employee name then it will update both department and if we update on bases of department name then also it will change department name of another employee like this update will lead to update anomaly.

3) Deletion Anomalies:

Deletion anomaly occurs when we are deleting unwanted data in the database but with that important data also get deleted.

For Example:

Project P02 is canceled and we need to delete data of project P02. If we will delete project P02 then with that employee and department related information will also be gone.

Thus, Anomalies and its type could be explained.

MAITRI K. HINGU

59. Disadvantages of Conventional or Traditional File Processing System.

OR

Drawbacks of Conventional or Traditional File Processing System.

OR

Cons of Conventional or Traditional File Processing System.

There are number of major disadvantages in conventional or traditional file processing system which are,

1) Data Redundancy and Inconsistency:

Same Information may be duplicated in several places because, in conventional file processing system, there are various files and different application programs.

All copies may not be updated properly because when we update data at one place and forget to change it at other place that time database transferred into inconsistent state and this inconsistency occurs due to data redundancy.

2) Difficulty in accessing data:

May have to write new application program to satisfy an unusual request because in conventional file processing system there are different files and different application program in which data is stored.

When we want to get specific data that time we must check all the files manually but it is time

MAITRI K. HINGU

consuming task another option is write new function that will return desired data. But every time for new need we must write new function and every application program has different access method.

The point here is that conventional file-processing environments do not allow needed data to be retrieved in a convenient and efficient manner.

3) Data Isolation:

Because data are scattered in various files, and files are in different format, writing new application programs every time to retrieve data is not good and data retrieval is difficult.

4) Integrity problems:

The data values stored in database must follow some consistency constraints. Developers enforce this constraint in programs. However, when new constraint is added, it is difficult to change the programs to enforce them when constraints involve several data items from different files.

5) Atomicity problem:

Atomicity is required to save the data values, it means that information is completely entered or canceled at all. Any system may fail at any time and at that time it is desired that data should be in a consistent state.

MAITRI K. HINGU

For Example: If you are buying a ticket from railway and you are in the process of money transaction. Suddenly, your internet got disconnected then you may or may not have paid for the ticket. If you have paid, then your ticket will be booked and if not then you will not be charged anything. That is called consistent state, means you have paid or not.

Same atomicity is not present in File Processing System.

6) Concurrent-access anomalies:

Many systems allow multiple user interface for simultaneous data access. But, in some situation simultaneous data access result into inconsistent data.

There is no protection or rules or security is provided for simultaneous data access.

For Example: Customer A and B are withdrawing money form the same account at the same time. Account has $500 in it, and they withdraw $100 and $50 respectively. So, in file processing system balance after transaction can be $350 or $400 or $450.

7) Security problems:

Not every user of the database system should be able to access all the data.

MAITRI K. HINGU

For Example: In university, payroll personnel must only handle payroll data and must not see academic records.

It is not possible to enforce this type of security constraint on File processing system.

Thus, Disadvantages of conventional file processing system could be explained.

MAITRI K. HINGU

60. Advantages of Database Management System or DBMS.
OR
Need of Database Management System or DBMS.
OR
Pros of Database Management System or DBMS.

Following are the advantages of the DataBase Management System (DBMS),

1) No Data Redundancy and Inconsistency:

In DBMS, all data or record are resided in one file and the application program is also same. So, we don't need to write the same data twice.

We are not writing the data twice so data redundancy will not be there. If data redundancy is not there then data inconsistency will also not occur.

2) No Difficulty in accessing data:

All record and data are resided in one file and there is only one application for accessing this type of file. So, there will not be problem of accessing data from different file and different application programs.

3) No Data Isolation:

There are not different files and different application programs. So, data is never going to be scattered in various files which results into no data isolation.

MAITRI K. HINGU

4) No Integrity problems:

Database will be in consistent state and DBMS will satisfy all integrity constraints like, Domain Integrity, Referential Integrity, Assertion, Authorization, etc. So, there will not be any integrity problem.

5) No Atomicity problem:

Atomic means to do the execution entirely or not to do at all.

We are working with machine; may be some time it breaks down or crashed due to any reason. That time it will perform the task entirely or will not do at all.

For Example: If you are buying a ticket from railway and you are in the process of money transaction. Suddenly, your internet got disconnected then you may or may not have paid for the ticket. If you have paid, then your ticket will be booked and if not then you will not be charged anything. That is called consistent state, means you have paid or not.

6) No Concurrent-access anomalies:

DBMS allow multiple user interface for simultaneous data access. But, in some situation simultaneous data access result into inconsistent data.

Protection or rules or security is provided for simultaneous data access.

MAITRI K. HINGU

For Example: Customer A and B are withdrawing money form the same account at the same time. Account has $500 in it, and they withdraw $100 and $50 respectively. So, in DBMS balance after transaction will be $350.

7) No Security problems:

Not every user of the database system should be able to access all the data.

For Example: In university, payroll personnel must only handle payroll data and must not see academic records.

It is possible to enforce this type of security constraint on DBMS.

Thus, Advantages of DataBase Management System (DBMS) could be explained.

MAITRI K. HINGU

61. What is Normalization? Explain it in detail.

Normalization:

"Normalization is a process of organizing data in such a way that it removes anomalies and data redundancy."

Normalization involves separating a database into tables and defining relationship between those tables.

There are three main stages of normalization called normal forms.

Each of them increases the level of normalization.

1) First Normal Form:

"Your relational schema R said to be in 1NF if and only if, every column of table in database has only atomic values in it."

For Example: Suppose a company wants to store the names and contact details of its employees. It creates a table that looks like this:

Name	DOB	Address	Contact
Patel Mehul Hareshbhai	2/1/88	Res Course Road, Baroda.	1234567890, 1357924680
Disha Shah	3/4/90	Akota, Baroda	4567890123
Nisha	5/6/86	Ahmedabad	7890654321, 2345678901

MAITRI K. HINGU

Here, in this example contact column has multiple valued as well as some employee name are in full name or only first name and last name or only first name. In address field, also. This table is not in the 1NF because first normal form says that every column must have atomic values in it.

Let's do some changes in the table to bring it into 1NF.

Table Name: EmployeeMst

First Name	Middle Name	Last Name	DOB	Address	City	Contact
Raj	Haresh bhai	Patel	2/1/88	Res Course Road.	Vadodara	1234567890
Raj	Haresh bhai	Patel	2/1/88	Res Course Road.	Vadodara	1357924680
Disha	Ashok bhai	Shah	3/4/90	Akota	Vadodara	4567890123
Nisha	Rasik bhai	Dave	5/6/86	M.G. Road.	Ahmedabad	7890654321
Nisha	Rasik bhai	Dave	5/6/86	M.G. Road.	Ahmedabad	2345678901

MAITRI K. HINGU

2) Second Normal Form:

"Your relational schema R said to be in 2NF if and only if, it is in 1NF and there is no partial functional dependency between any attribute."

For Example: Employees in the company can handle more than one project as well as for handling one project we need more than one employee. Consider following table for above scenario.

Table Name: EmployeeMst

First Name	Middle Name	Last Name	DOB	Address	City	Contact	Project
Raj	Haresh bhai	Patel	2/1/88	Res Course Road.	Vadodara	1234567890	Project 01
Raj	Haresh bhai	Patel	2/1/88	Res Course Road.	Vadodara	1357924680	Project 02
Disha	Ashok bhai	Shah	3/4/90	Akota	Vadodara	4567890123	Project 02
Nisha	Rasik bhai	Dave	5/6/86	M.G. Road.	Ahmedabad	7890654321	Project 01
Nisha	Rasik bhai	Dave	5/6/86	M.G. Road.	Ahmedabad	2345678901	Project 02

MAITRI K. HINGU

Project – to – Employee

Many – to – Many

Above table is in 1NF but because of Many – to – Many relationship between Employee and project We need to make another table that is ProjectMst and as well as EmployeeProjectDetail

Table Name: EmployeeMst

EID	First Name	Middle Name	Last Name	DOB	Address	City	Contact
E01	Raj	Haresh bhai	Patel	2/1/88	Res Course Road.	Vadodara	1234567890
E01	Raj	Haresh bhai	Patel	2/1/88	Res Course Road.	Vadodara	1357924680
E02	Disha	Ashok bhai	Shah	3/4/90	Akota	Vadodara	4567890123
E03	Nisha	Rasik bhai	Dave	5/6/86	M.G. Road.	Ahmedabad	7890654321
E03	Nisha	Rasik bhai	Dave	5/6/86	M.G. Road.	Ahmedabad	2345678901

MAITRI K. HINGU

Table Name: ProjectMst

PID	ProjectName	Location
P01	Project01	Baroda
P02	Project02	Ahmedabad

Table Name: EmployeeProjectDetail

EPID	EID	PID
EP01	E01	P01
EP02	E01	P02
EP03	E02	P02
EP04	E03	P01
EP05	E03	P02

3) Third Normal Form:

"Your relational schema R said to be in 3NF if and only if, it is in 2NF and all non-prime attributes are fully functionally dependent on prime attributes."

Attributes which are not dependent on prime attributes must be eliminated.

For Example: Consider above tables which we have normalized in 2NF. Those table does not have prime attribute. We need to create one for each table.

EmployeeMst table is in 2NF but there is no primary key in table and because of multivalued attribute, that is contact number, all data are repeated.

MAITRI K. HINGU

We need to find relationship between employee and contact number that is one to many. So, we need to create another table that is ContactMst.

Table Name: EmployeeMst

EID	FName	MName	LName	DOB	Address	City
E01	Mehul	Haresh bhai	Patel	2/1/88	Res Course Road.	Baroda
E02	Disha	Ashok bhai	Shah	3/4/90	Akota	Baroda
E03	Nisha	Rasik bhai	Dave	5/6/86	M.G. Road.	Ahmed abad

Table Name: ContactMst

CNID	Contact	EID
CN01	1234567890	E01
CN02	1357924680	E01
CN03	4567890123	E02
CN04	7890654321	E03
CN05	2345678901	E03

Here, EID in EmployeeMst is primary key and in ContactMst is foreign key. CNID in ContactMst is Primary key. So, each attribute in both table is dependent on Primary key and we can say that database is in 3NF.

MAITRI K. HINGU

4) Boyce-Codd Normal Form:

"Your relational schema R said to be in BCNF if and only if, it is in 3NF and all its functional dependency FD: $X \rightarrow Y$ one of the following is true.

1) $X \rightarrow Y$ is a trivial functional dependency ($Y \subseteq X$)
2) X is a super key for schema R"

Thus, Normalization could be explained.

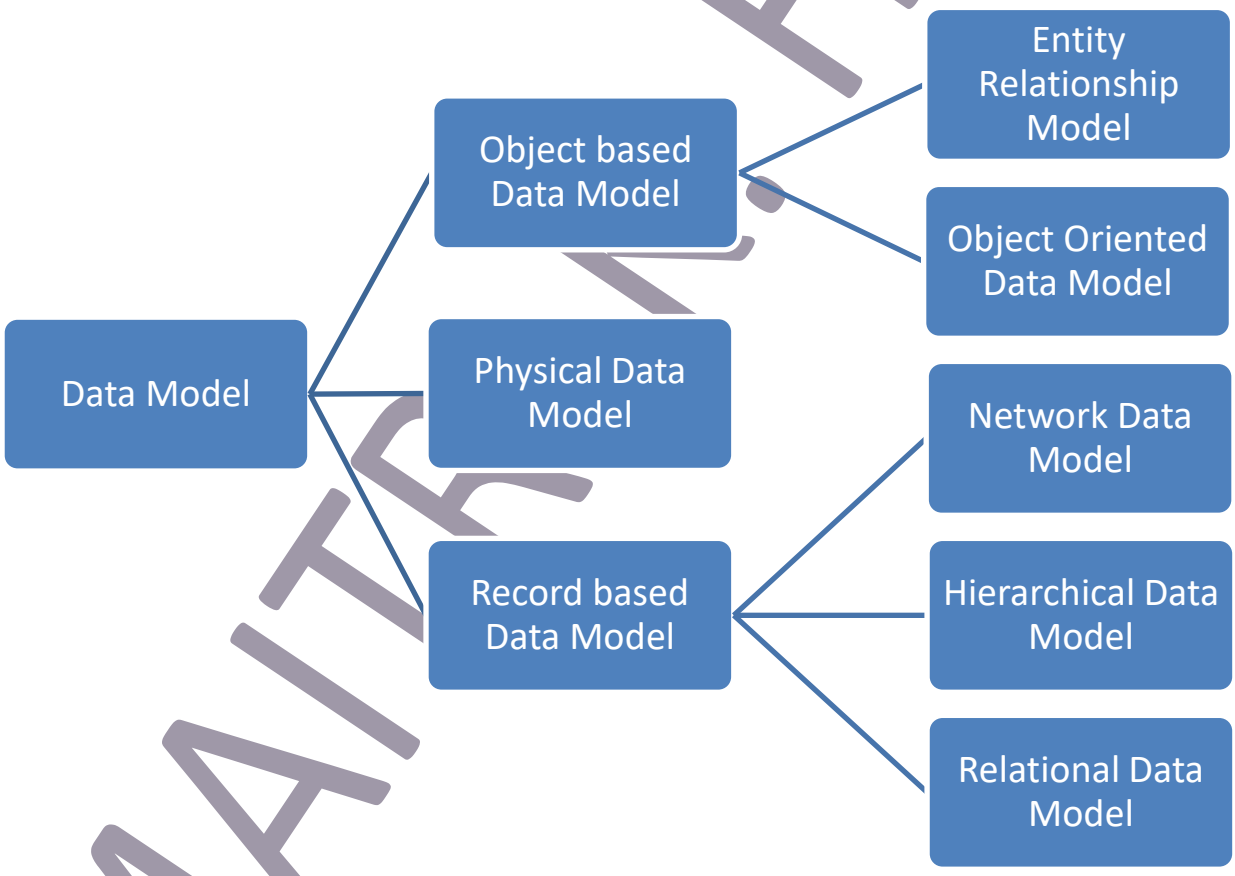
MAITRI K. HINGU

62. What are Data Models?

"A database model is a type of data model that determines the logical structure of a database and fundamentally determines in which manner data can be stored, organized and manipulated."

Depending on the levels of data we are modeling, we have divided data models into 3 categories – Object Based, Physical and Record based Data models.

The most popular example of a database model is the relational model, which uses a table-based format.



Thus, Data models could be explained.

MAITRI K. HINGU

63. Explain Record based data models in detail.

There are 3 types of record based data models defined so far- Hierarchical, Network and Relational data models. Most widely used record based data model is relational data model. Other two are not widely used. Let us understand how they are different from each other.

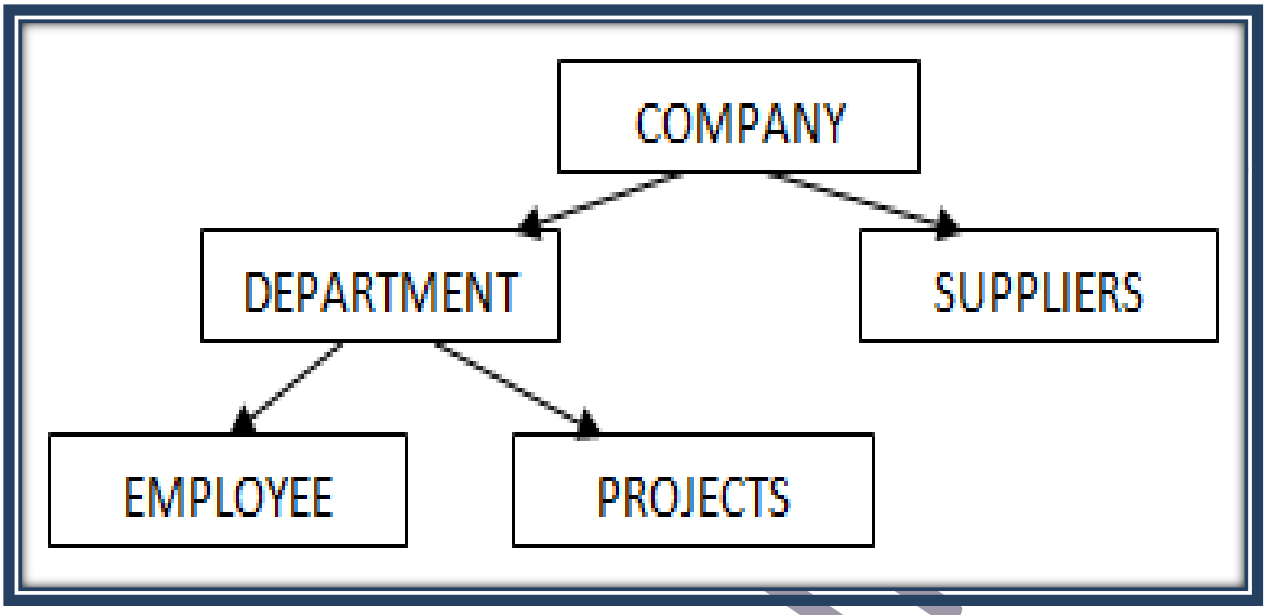
1) Hierarchical Data Model:

Imagine we have to create a database for a company. What are the entities involved in it? Company, its department, its supplier, its employees, different projects of the company etc. are the different entities we need to take care of. If we observe each of the entity they have parent –child relationship. We can design them like we do ancestral hierarchy. In our case, Company is the parent and rests of them are its children. Department has employees and project as its children and so on. This type of data modeling is called hierarchical data model.

In this data model, the entities are represented in a hierarchical fashion. Here we identify a parent entity, and its child entity. Again we drill down to identify next level of child entity and so on. This model can be imagined as folders inside a folder.

In our example above, it is diagrammatically represented as below:

MAITRI K. HINGU

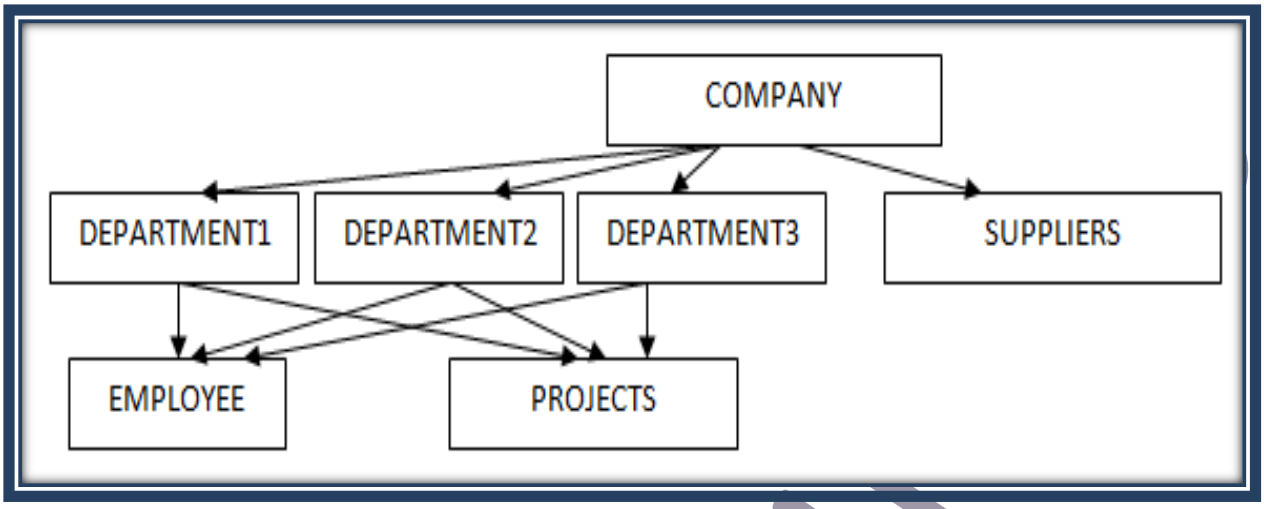


It can also be imagined as root like structure. This model will have only one main root. It then branches into sub-roots, each of which will branch again. This type of relationship is best defined for 1:M type of relationships.

For Example:

One company has multiple departments (1:M), one company has multiple suppliers (1:M), one department has multiple employees (1:M), each department has multiple projects(1:M). If we have M:N relationships, then we have to duplicate the entities and show it in the diagram. For example, if a project in the company involves multiple departments, then our hierarchical representation changes as below:

MAITRI K. HI



Advantages:

- It helps to address the issues of flat file data storage. In flat files, data will be scattered and there will not be any proper structuring of the data. This model groups the related data into tables and defines the relationship between the tables, which is not addressed in flat files.

Disadvantages:

- Redundancy: When data is stored in a flat file, there might be repetition of same data multiple times and any changes required for the data will need to change in all the places in the flat file. Missing to update at any one place will cause incorrect data. This kind redundancy is solved by hierarchical model to some extent. Since records are grouped under related table, it solves the flat file redundancy issue. But look at the many to many relationship examples given above. In such case, we have to store same project information for more than one department. This is duplication of data and hence a

MAITRI K. HI

redundancy. So, this model does not reduce the redundancy issue to a significant level.

- As we have seen above, it fails to handle many to many relationships efficiently. It results in redundancy and confusion. It can handle only parent-child kind of relationship.
- If we need to fetch any data in this model, we have to start from the root of the model and traverse through its child till we get the result. In order to perform the traversing, either we should know well in advance the layout of model or we should be very good programmer. Hence fetching through this model becomes bit difficult.
- Imagine company has got some new project details, but it did not assign it to any department yet. In this case, we cannot store project information in the PROJECT table, till company assigns it to some department. That means, to enter any child information, its parent information should be already known / entered.

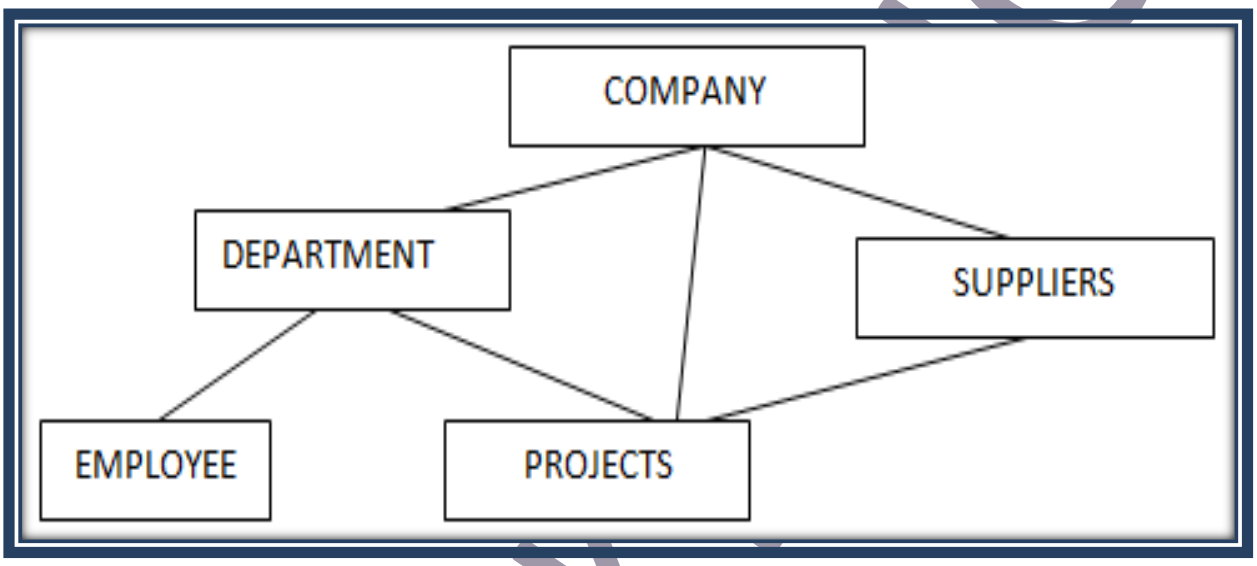
2) Network Data Models:

This is the enhanced version of hierarchical data model. It is designed to address the drawbacks of the hierarchical model. It helps to address M:N relationship. This data model is also represented as hierarchical, but this model will not have single parent concept. Any child in the tree can have multiple parents here.

Let us revisit our company example. A company has different projects and departments in the company own those projects. Even suppliers of the company give input

MAITRI HINGU

for the project. Here Project has multiple parents and each department and supplier have multiple projects. This is represented as shown below. Basically, it forms a network like structure between the entities, hence the name.



Advantages:

- Accessing the records in the tables is easy since it addresses many to many relationships. Because of this kind of relationship, any records can be easily pulled by using any tables. For example, if we want to know the department of project X and if we know SUPPLIER table, we can pull this information. i.e.; SUPPLIER has the information about project X which includes the departments involved in the projects too. Hence makes the accessibility to any data easier, and even any complex data can be retrieved easily and quickly.
- Because of the same feature above, one can easily navigate among the tables and get any data.

MAITRI K. HINGU

- It is designed based on database standards – ANSI/SP ARC.

Disadvantages:

- If there is any requirement for the changes to the entities, it requires entire changes to the database. There is no independence between any objects. Hence any changes to the any of the object will need changes to the whole model. Hence difficult to manage.
- It would be little difficult to design the relationship between the entities, since all the entities are related in some way. It requires thorough practice and knowledge about the designing.

3) Relational Data Models:

This model is designed to overcome the drawbacks of hierarchical and network models. It is designed completely different from those two models. Those models define how they are structured in the database physically and how they are inter-related. But in the relational model, we are least bothered about how they are structured. It purely based on how the records in each table are related. It purely isolates physical structure from the logical structure. Logical structure is defining records are grouped and distributed.

Let us try to understand it by an example. Let us consider department and employee from our previous examples above. In this model we look at employee with

MAITRI K. HINGU

its data. When we say an employee what all comes into our mind? His employee id, name, address, age, salary, department that he is working etc. are attributes of employee. That means these details about the employee forms columns in employee table and value set of each employee for these attribute forms a row/record for an employee. Similarly, department has its id, name.

Now, in the employee table, we have column which uniquely identifies each employee – that is employee Id column. This column has unique value and we are able to differentiate each employee from each other by using this column. Such column is called as primary key of the table. Similarly department table has DEPT_ID as primary key. In the employee table, instead of storing whole information about his department, we have DEPT_ID from department table stored. i.e.; by using the data from the department table, we have established the relation between employee and department tables.

Table Name: DepartmentMst

DID	DName	DCode
1	HR	D01
2	Finance	D02
3	Marketing	D03

Table Name: EmployeeMst

EID	EName	Address	Contact	DID
1	Nisha	M. G. Road	1234567890	1

MAITRI K. HINGU

2	Raj	Akota	2345678901	2
3	Disha	Alkapuri	3456789012	3
4	Kayaan	Res Course Road	4567890123	3
5	Hema	Gandhinagar	5678901234	1

Observe the table structures above. They are very simple to understand. There is no redundant data as well. It addressed major drawback of earlier data models. This type of data model is called relational data model.

This model is based on the mathematical concepts of set theory. It considers the tables as a two dimensional table with rows and columns. It is least bothered about the physical storage of structure and data in the memory. It considers only the data and how it can be represented in the form of rows and columns, and the way it can establish the relation between other tables.

A relational data model revolves around 5 important rules.

1. Order of rows / records in the table is not important. For example, displaying the records for Joseph is independent of displaying the records for Rose or Mathew in Employee table. It does not change the meaning or level of them. Each record in the table is independent of other. Similarly, order of columns in the table is not important. That means, the value in each column for a record is independent of other. For example, representing DEPT_ID

MAITRI K. HINGU

at the end or at the beginning in the employee table does not have any affect.

2. Each record in the table is unique. That is there is no duplicate record exists in the table. This is achieved by the use of primary key or unique constraint.
3. Each column/attribute will have single value in a row. For example, in Department table, DEPT_NAME column cannot have 'Accounting' and 'Quality' together in a single cell. Both has to be in two different rows as shown above.
4. All attributes should be from same domain. That means each column should have meaningful value. For example, Age column cannot have dates in it. It should contain only valid numbers to represent individual's age. Similarly, name columns should have valid names, Date columns should have proper dates.
5. Table names in the database should be unique. In the database, same schema cannot contain two or more tables with same name. But two tables with different names can have same column names. But same column name is not allowed in the same table.

Advantages:

- Structural independence: Any changes to the database structure, does not the way we are accessing the data. For example, Age is added to Employee table. But it does not change the relationship between the other tables nor changes the existing data. Hence it provides the total independence from its structure.

MAITRI K. HINGU

- Simplicity: This model is designed based on the logical data. It does not consider how data are stored physically in the memory. Hence when the designer designs the database, he concentrates on how he sees the data. This reduces the burden on the designer.
- Because of simplicity and data independence, this kind of data model is easy to maintain and access.
- This model supports structured query language – SQL. Hence it helps the user to retrieve and modify the data in the database. Using SQL, user can get any specific information from the database.

Disadvantages:

Compared to the advantages above, the disadvantages of this model can be ignored.

- High hardware cost: To separate the physical data information from the logical data, more powerful system hardware's memory is required. This makes the cost of database high.
- Sometimes, design will be designed till the minute level, which will lead to complexity in the database.

Thus, Record based data model could be explained.

MAITRI K. HINGU

64. Differentiate Record based Data Model

Below table provides the comparison among the three record based Data Models

Hierarchical Data Model	Network Data Models	Relational Data Models
Supports One-Many Relationship	Supports both one to many and Many to Many relationship	Supports both one to many and Many to Many relationship
Because of single parent-child relationship, difficult to navigate through the child	It establishes the relationship between most of the objects, hence easy to access compared to hierarchical model	It provides SQL, which makes the access to the data simpler and quicker.
Flexibility among the different object is restricted to the child.	Because of the mapping among the sub level tables, flexibility is more	Primary and foreign key constraint makes the flexibility much simpler than other models.
Based on the physical storage details	Based on the physical storage details	Based on the logical data view

Thus, Record based data model could be differentiated.

MAITRI K. HINGU

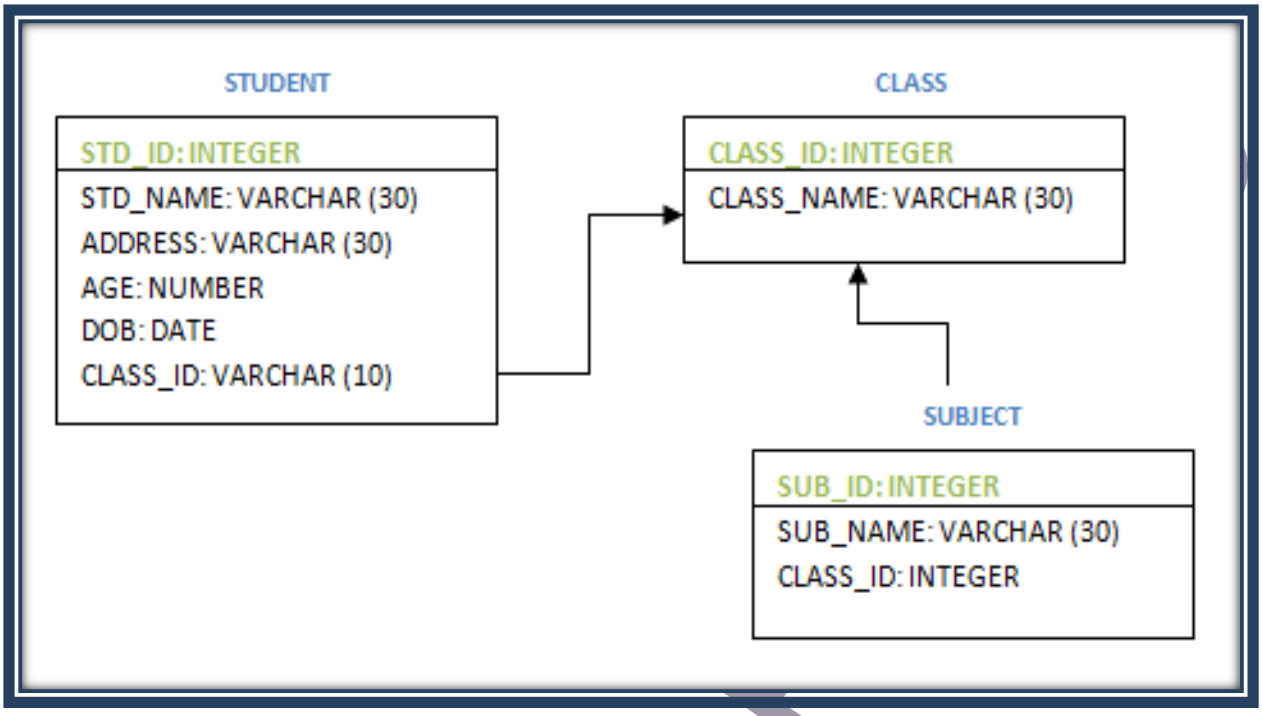
65. Explain Physical data model in detail.

"Physical data model is a data model which describes "How data are stored in computer memory?", "How they are scattered and ordered in the memory?", and "How they would be retrieved from memory?" Basically, physical data model represents the data at physical level."

we can say a physical data model has,

- Tables and its specifications – table names and their columns. Columns are represented along with their datatypes and size. In addition primary key of each table is shown at the top of the column list.
- Foreign keys are used to represent the relationship between the tables. Mapping between the tables are represented using arrows between them.
- Physical data model can have de-normalized structure based on the user requirement. The tables might not be in normalized forms.

MAITRI K. HINGU



Thus, Physical data model could be explained.

66. Explain Object based data model in detail.

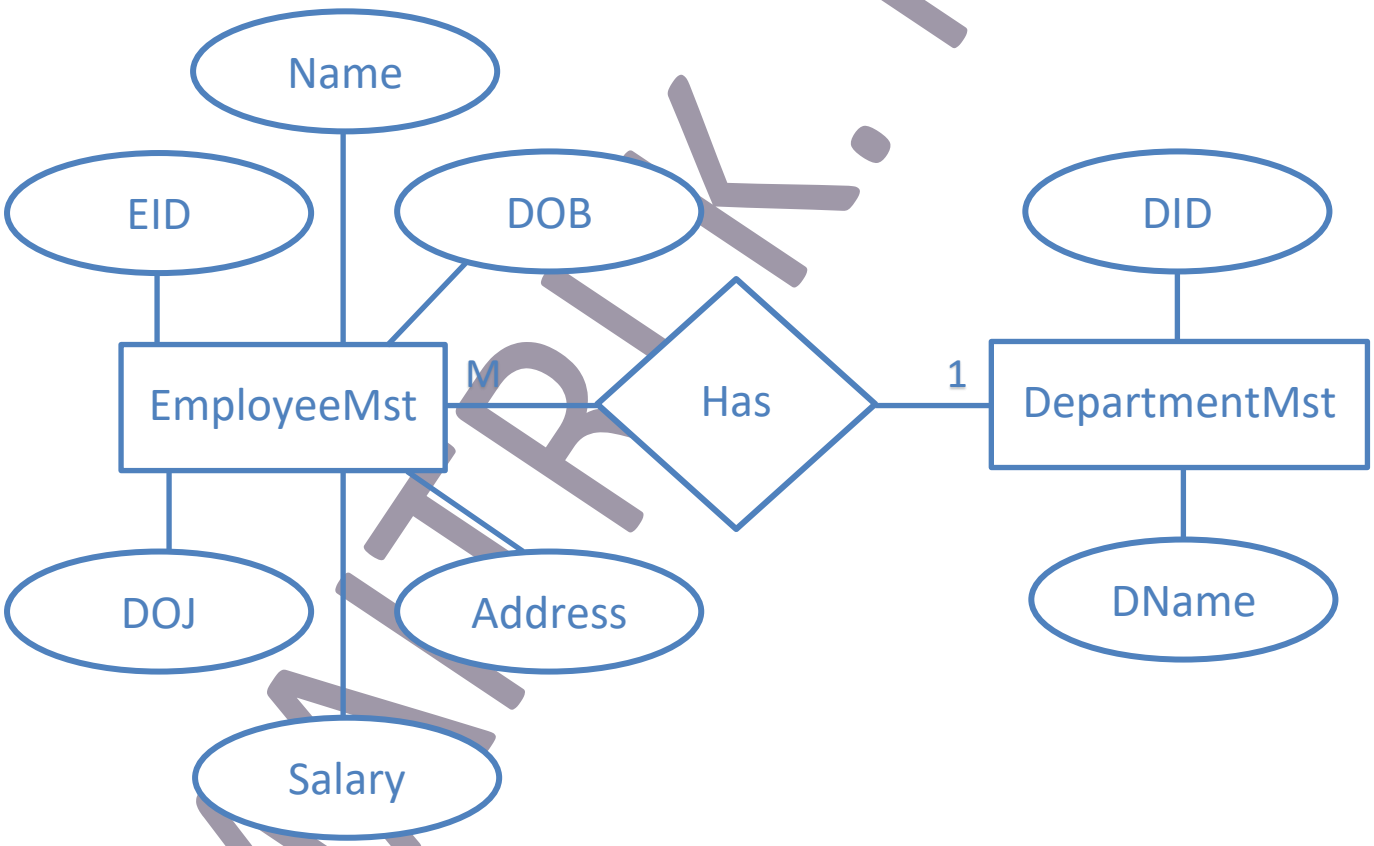
Object based Data Models are designed using the entities in the real world, attributes of each entity and their relationship. It picks up each thing/object in the real world which is involved in the requirement.

There are two types of object based data Models – Entity Relationship Model and Object oriented data model. ER data model is one of the important data model which forms the basis for the all the designs in the database world. It defines the mapping between the entities in the database. Object oriented data model, along with the mapping between the entities, describes the state of each entity and the tasks performed by them.

MAITRI K. H

1) Entity Relationship Data Model:

Entities or real world objects are represented in a rectangular box. Their attributes are represented in ovals. Primary keys of entities are underlined. All the entities are mapped using diamonds. Basically, ER model is a graphical representation of real world objects with their attributes and relationship. It makes the system easily understandable. This model is considered as a top down approach of designing a requirement.



MAITRI K. HINGU

Advantages:

- It makes the requirement simple and easily understandable by representing simple diagrams.
- One can convert ER diagrams into record based data model easily.
- Easy to understand ER diagrams

Disadvantages:

- No standard notations are available for ER diagram. There is great flexibility in the notation. It's all depends upon the designer, how he draws it.
- It is meant for high level designs. We cannot simplify for low level design like coding.

2) Object Oriented Data Models:

This data model is another method of representing real world objects. It considers each object in the world as objects and isolates it from each other. It groups its related functionalities together and allows inheriting its functionality to other related sub-groups.

For Example: Consider EmployeeMst Table which we will convert into class.

public class EmployeeMst

MAITRI K. HINGU

```
{
  int EID;
  string FName;
  string MName;
  string LName;
  string Address;
  string City;
  string State;
  string Country;
  int Pincode;
  int Salary;
  string EmailID;
  date DOJ;

  public void getData()
  {
  ……..
  }
  public void putData()
  {
    ……..
  }

} e1, e2, e3;

 e1.getData();
 e2.getData();
 e3.getData();
 e1.putData();
 e2.putData();
 e3.putData();
```

MAITRI K. HINGU

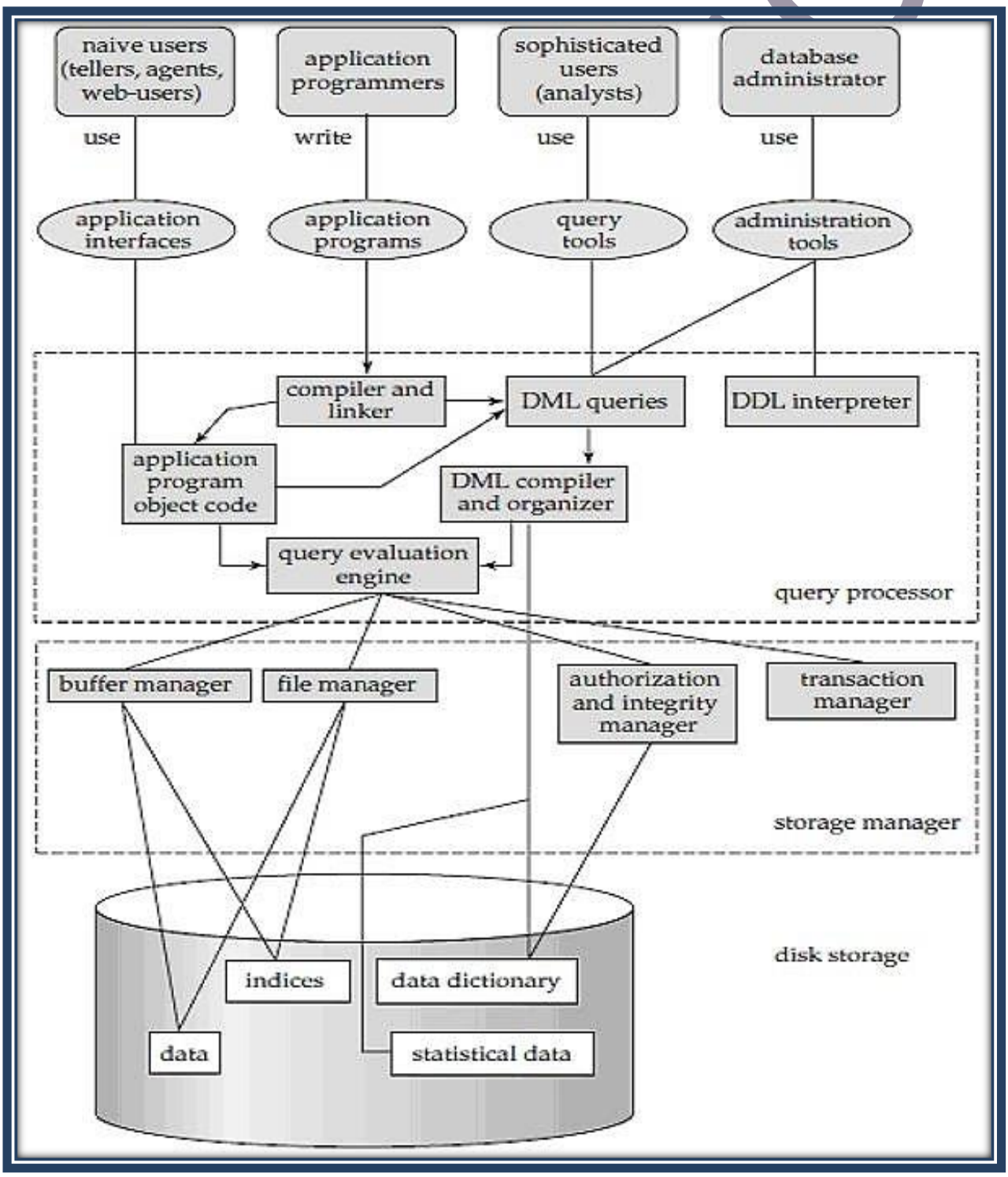
Here, e1, e2, e3 will work as row or record, class will work as table and data member will work as column and member method will work as DML operation.

Thus, object-based data model could be explained.

MAITRI K. HINGU

67. Explain Database system structure in detail.

Database system are partitioned into modules for different functions. Some functions may be provided by the operating system.



1) Query Processor:

- **DDL Interpreter:** It interprets DDL statements and records the definitions in the data dictionary.
- **DML Compiler:** It translates DML statements in query language into an evaluation plan consisting of low level instruction that the query evaluation engine understands.
- **Query Evaluation Engine:** It executes low level instruction generated by the DML compiler.

2) Storage Manager:

- **Authorization and Integrity Manager:** It checks for the satisfaction of integrity constraints as well as authority of users to access data.
- **Transaction Manager:** It ensures that the database remains in a consistent state despite system failures and simultaneous transaction execution proceed without conflicting.
- **File Manager:** It manages the allocation of space on disk storage and the data structures used to represent information stored on disk.
- **Buffer Manager:** It is responsible for fetching data from disk storage into main memory. It also decides what data to fetch into main memory. The buffer manager is the crucial part of the database system.

The storage manager implements several data structures as part of the physical system implementation.

MAITRI K. HINGU

- **Data File:** It stores the database itself.
- **Data Dictionary:** It stores metadata about the structure of the database schema of the database.
- **Indices:** For speedy access of data indices provides pointer to the value which is holding by the data item.

Thus, Database System Structure could be explained.

68. Explain Types of Database Users in detail.

Database users:

"They are the one who really use and take the benefits of database. There will be different types of users depending on their need and way of accessing the database."

1) **Application Programmers:** They are the developers who interact with the database by means of DML queries. These DML queries are written in the application programs like C, C++, JAVA, Pascal etc. These queries are converted into object code to communicate with the database. For example, writing a C program to generate the report of employees who are working department will involve a query to fetch the data from database. It will include an embedded SQL query in the C Program.

2) **Sophisticated Users:** They are database developers, who write SQL queries to select/insert/delete/update data. They do not use any application or programs to

MAITRI K. HINGU

request the database. They directly interact with the database by means of query language like SQL. These users will be scientists, engineers, analysts who thoroughly study SQL and DBMS to apply the concepts in their requirement. In short, we can say this category includes designers and developers of DBMS and SQL.

3) **Specialized Users:** These are also sophisticated users, but they write special database application programs. They are the developers who develop the complex programs to the requirement.

4) **Stand – alone Users:** These users will have stand – alone database for their personal use. These kinds of database will have readymade database packages which will have menus and graphical interfaces.

5) **Naive Users:** these are the users who use the existing application to interact with the database. For example, online library system, ticket booking systems, ATMs etc. which has existing application and users use them to interact with the database to fulfill their requests.

Thus, Types of Database users could be explained.

MAITRI K. HINGU

69. Who is Database Administrator? Explain duties of Database Administrator.

DBA (DataBase Administrator):

"The DBA is a person having central control over data and programs accessing that data."

1) Installing and upgrading the database server and application tools
2) Allocating system storage and planning future storage requirements for the database system
3) Modifying the database structure, as necessary, from information given by application developers
4) Enrolling users and maintaining system security
5) Ensuring compliance with database vendor license agreement
6) Controlling and monitoring user access to the database
7) Monitoring and optimizing the performance of the database
8) Planning for backup and recovery of database information
9) Maintaining archived data
10) Backing up and restoring databases
11) Contacting database vendor for technical support
12) Generating various reports by querying from database as per need

Thus, DBA and his duties could be explained.

MAITRI K. HINGU

70. What is E-R Diagram? Explain its notation in detail.

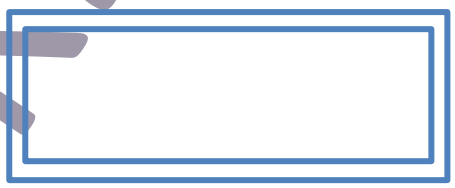
"E-R Diagram is based on real world that consist of basic objects called entities and of relationship among these objects. Entities are described in a database by asset of attributes."

Notation / Symbol:

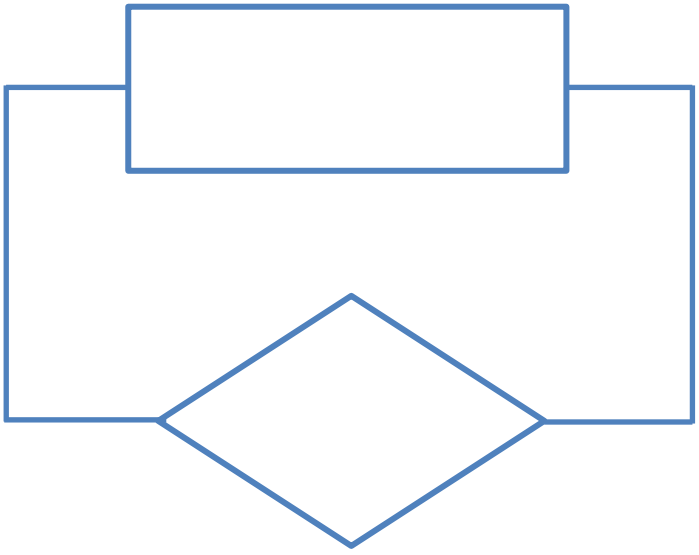
1) Entity / Strong Entity:



2) Weak Entity:



3) Recursive Entity:

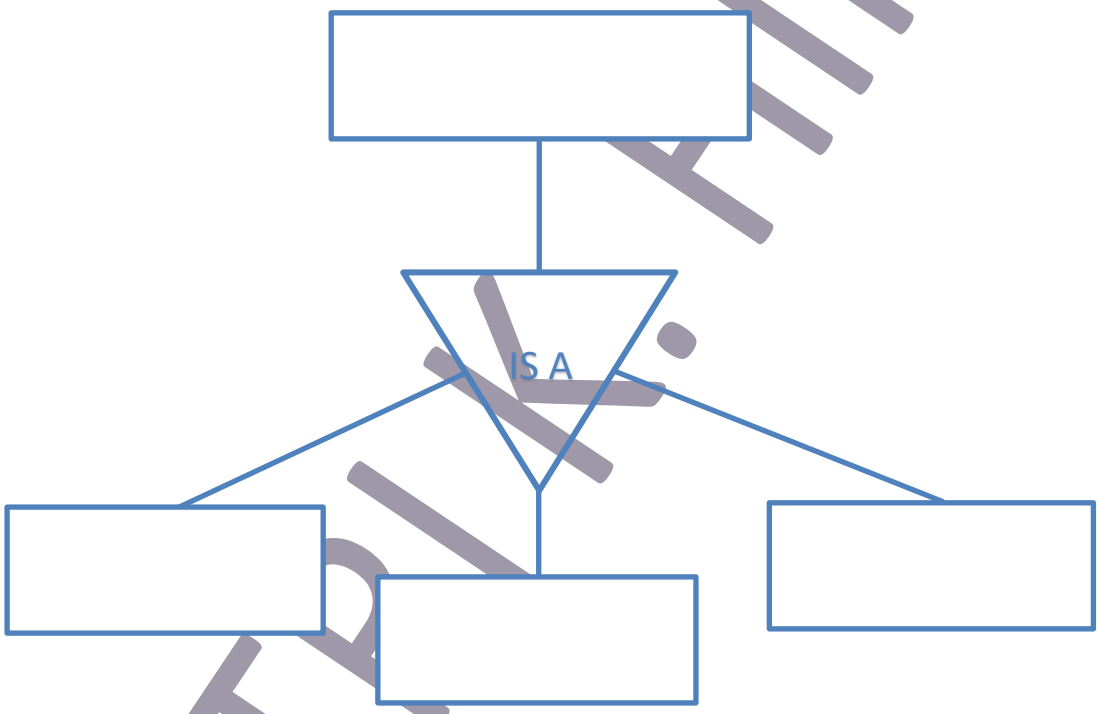


MAITRI K. HINGU

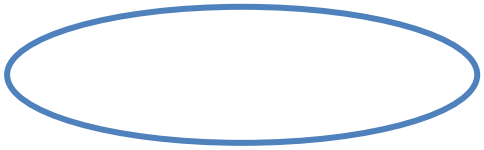
4) Composite Entity:



5) Super Type Entity & Sub Type Entity:

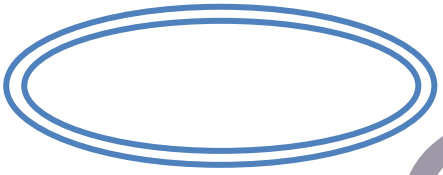


6) Attribute / Single valued attribute / Simple Attribute / Complex Attribute / Stored Attribute / Non-Prime or Non-Key Attribute / Required Attribute / Optional Attribute:



MAITRI K. HINGU

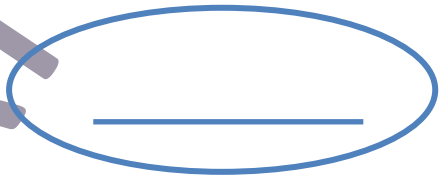
7) Multivalued Attribute:



8) Composite or Compound Attribute:



9) Prime or Key Attribute:

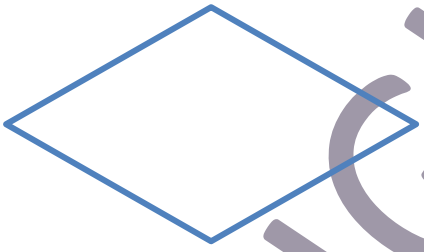


10) Derived Attribute:

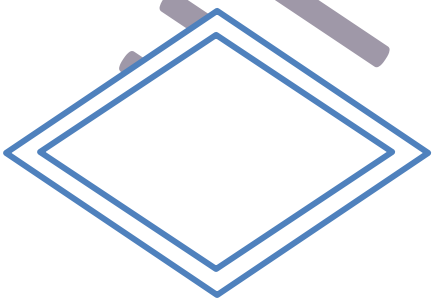


MAITRI K. HINGU

11) Relationship / Strong Relationship:



12) Weak Relationship:



13) One – to – One Relationship Connector:

1 1

14) One – to – Many Relationship Connector:

1 M

15) Many – to – One Relationship Connector:

M 1

16) Many – to – Many Relationship Connector:

M M

Thus, E-R Diagram and its notation could be explained.

MAITRI K. HINGU

71. What is Relational Algebra? Explain it in detail.

Relational Algebra is procedural query language, which takes Relation as input and generate relation as output. Relational algebra mainly provides theoretical foundation for relational databases and SQL.

Unary Relational Operations

1. SELECT (Symbol: σ)
2. PROJECT (Symbol: π)
3. RENAME (Symbol: ρ)

Relational Algebra Operations from Set Theory

4. UNION (Symbol: ∪)
5. INTERSECTION (Symbol: ∩)
6. DIFFERENCE (Symbol: –)
7. CARTESIAN PRODUCT (Symbol: X)

Binary Relational Operations

8. JOIN (Symbol: ⋈)

Consider the below table for Every Example.

EmployeeMst (EID, FName, MName, LName, Address, City, Pincode, Contact, DOB, DOJ, Salary)

MAITRI K. HINGU

1. SELECT (σ)

The SELECT operation is used for selecting a subset of the tuples according to a given selection condition. Sigma(σ)Symbol denotes it. It is used as an expression to choose tuples which meet the selection condition. Select operator selects tuples that satisfy a given predicate.

σ p (r)

σ is the predicate

r stands for relation which is the name of the table

p is prepositional logic

Example 1:

σ City = "Surat" (EmployeeMst)

Output will be same as SELECT * FROM EmployeeMst WHERE City='Surat'.

Example 2:

σ City = "Surat" and Pincode=395006 (EmployeeMst)

Output will be same as SELECT * FROM EmployeeMst WHERE City='Surat' AND Pincode=395006.

Example 3:

σ Salary>50000 (EmployeeMst)

Output will be same as SELECT * FROM EmployeeMst WHERE Salary>50000.

MAITRI K. HINGU

2. Projection(π)

The projection eliminates all attributes of the input relation but those mentioned in the projection list. The projection method defines a relation that contains a vertical subset of Relation.

This helps to extract the values of specified attributes to eliminates duplicate values. π (pi) symbol is used to choose attributes from a relation. This operator helps you to keep specific columns from a relation and discards the other columns.

Example:

π City (EmployeeMst)

Output will be same as SELECT DISTINCT City FROM EmployeeMst.

3. Rename (ρ)

Rename is a unary operation used for renaming attributes of a relation.

ρ (a/b)R will rename the attribute 'b' of relation 'R' by 'a'.

Example:

ρ (Date of Birth/DOB) EmployeeMst

MAITRI K. HINGU

4. Union operation (U)

UNION is symbolized by U symbol. It includes all tuples that are in tables A or in B. It also eliminates duplicate tuples. So, set A UNION set B would be expressed as: A U B

For a union operation A U B to be valid, the following conditions must hold

- A and B must have the same number of attributes.
- Attribute domains need to be compatible.
- Duplicate tuples should be automatically removed.

Example:

Table A

Col 1	Col 2
1	1
1	2

Table B

Col 1	Col 2
1	1
1	3

A U B gives

Col 1	Col 2
1	1
1	2
1	3

MAITRI K. HINGU

5. INTERSECTION Operation (∩)

INTERSECTION is symbolized by ∩ symbol. It includes all tuples that are in tables A AND in B. It also eliminates duplicate tuples. So, set A INTERSECTION set B would be expressed as: A ∩ B

For a INTERSECTION operation A ∩ B to be valid, the following conditions must hold

- A and B must have the same number of attributes.
- Attribute domains need to be compatible.
- Duplicate tuples should be automatically removed.

Example:

Table A

Col 1	Col 2
1	1
1	2

Table B

Col 1	Col 2
1	1
1	3

A ∩ B gives

Col 1	Col 2
1	1

MAITRI K. HINGU

6. Difference Operation (–)

It is denoted by – Symbol. The result of A – B, is a relation which includes all tuples that are in A but not in B.

For a Difference operation A – B to be valid, the following conditions must hold

- The attribute name of A has to match with the attribute name in B.
- The two-operand relations A and B should be either compatible or Union compatible.
- It should be defined relation consisting of the tuples that are in relation A, but not in B.

Example:

Table A

Col 1	Col 2
1	1
1	2

Table B

Col 1	Col 2
1	1
1	3

A – B

Col 1	Col 2
1	2

MAITRI K. HINGU

7. Cartesian product (X)

This type of operation is helpful to merge columns from two relations. Generally, a Cartesian product is never a meaningful operation when it performs alone. However, it becomes meaningful when it is followed by other operations.

Cross product between two relations let say A and B, so cross product between A X B will results all the attributes of A followed by each attribute of B. Each record of A will pairs with every record of B.

Example:

```
A                                   B
 (Name   Age  Sex )                 (Id   Course)
 -----------------                  ------------
 Ram    14   M                       1     DS
 Sona   15   F                       2     DBMS
 kim    20   M

  A X B
Name   Age   Sex   Id   Course
---------------------------------
Ram    14    M     1    DS
Ram    14    M     2    DBMS
Sona   15    F     1    DS
Sona   15    F     2    DBMS
Kim    20    M     1    DS
Kim    20    M     2    DBMS
```

Note: if A has ‘n’ tuples and B has ‘m’ tuples then A X B will have ‘n*m’ tuples.

8. Join Operations (⋈):

Join operation is essentially a cartesian product followed by a selection criterion. Join operation denoted by ⋈.

JOIN operation also allows joining variously related tuples from different relations.

Types of JOIN:

1. Inner Join (⋈)

2. Outer join

2.1. Left Outer Join (⟕)

2.2. Right Outer Join (⟖)

2.3. Full Outer Join (⟗)

1. Inner Join (⋈):

In an inner join, only those tuples that satisfy the matching criteria are included, while the rest are excluded.

Example:

Table A

Col 1	Col 2
1	1
1	2

Table B

Col 1	Col 2
1	1
1	3

A ⋈ A.Col2 = B.Col2 (B)

Col 1	Col 2
1	1

MAITRI K. HINGU

Output will be same as "SELECT * FROM A, B WHERE A.Col2 = B.Col2;"

2. Outer Join:

In an outer join, along with tuples that satisfy the matching criteria, we also include some or all tuples that do not match the criteria.

2.1. Left Outer Join (A ⟕ B):

In the left outer join, operation allows keeping all tuple in the left relation. However, if there is no matching tuple is found in right relation, then the attributes of right relation in the join result are filled with null values.

Example:

Table A

No	Square
2	4
3	9
4	16

Table B

No	Cube
2	8
3	27
5	125

A ⟕ A.Col2 = B.Col2 (B)

No	Square	Cube
2	4	8
3	9	27
4	16	-

MAITRI K. HINGU

2.2. Right Outer Join (A ⟖ B):

In the right outer join, operation allows keeping all tuple in the right relation. However, if there is no matching tuple is found in the left relation, then the attributes of the left relation in the join result are filled with null values.

Example:

Table A

No	Square
2	4
3	9
4	16

Table B

No	Cube
2	8
3	27
5	125

A ⟖ A.Col2 = B.Col2 (B)

No	Cube	Square
2	8	4
3	27	9
5	125	-

MAITRI K. HINGU

2.3. Full Outer Join (A ⟗ B):

In a full outer join, all tuples from both relations are included in the result, irrespective of the matching condition.

Example:

Table A

No	Square
2	4
3	9
4	16

Table B

No	Cube
2	8
3	27
5	125

A ⟗ A.Col2 = B.Col2 (B)

No	Square	Cube
2	4	8
3	9	27
4	16	-
5	-	125

MAITRI HINGU